

C Hash: The Chitù Hash Function

Cryptographic Hash Function Design Document

Liting Zhang¹ Ying Zhang^{2,3}

¹ Westone Cryptographic Research Center
CETC Cyberspace Security Technology CO., LTD.

`zhang.liting@westone.com.cn`

² Trusted Computing and Information Assurance Laboratory
Institute of Software, Chinese Academy of Sciences

³ University of Chinese Academy of Sciences

July 24, 2026

Table of Contents

1	Algorithm Description	5
1.1	Overview	5
1.2	The Rocket-JH Iteration	6
1.2.1	Symbols	6
1.2.2	Message Padding	7
1.2.3	The Rocket-JH Construction	7
1.2.4	Domain Separation: CTR-Perm and CTR-Func	8
1.2.5	The Lane-Dedicated Rapid Hash (LDRH)	9
1.3	The C-Engine Permutation	10
1.3.1	Algorithm Overview	10
1.3.2	Component Specifications	13
1.4	C Hash Parameters	14
2	Design Rationale	15
2.1	The Rationales of the Rocket-JH Mode	15
2.1.1	The Homogeneity Barrier	15
2.1.2	Structural Decoupling: The Compress-then-Randomize Paradigm	15
2.1.3	Temporal Decoupling: Pairwise Distinct Round Functions	16
2.1.4	From JH to Rocket-JH: CTR-Perm and CTR-Func	16
2.1.5	Latency-Aware Decoupling: The LDRH Fast Path	16
2.2	The Rationales of C-Engine	17
2.2.1	The Wide-Permutation Dilemma	17
2.2.2	The Composition Paradigm	17
2.2.3	The Design Rationales of Components	17
2.3	The Criterion for Parameter Selection	19
3	Security Statements and Analyses	22
3.1	Concrete Security Claims	22
3.2	Provable Security Analysis	24
3.2.1	Indifferentiability Bounds	24
3.2.2	Hash Security Guarantees	25
3.3	Security Analysis of C-Engine Permutation	26
3.3.1	Security Analysis of Components	26
3.3.2	Differential Cryptanalysis	26
3.3.3	Linear Cryptanalysis	27
3.3.4	Zero-Sum Distinguisher	27
3.3.5	Algebraic Attacks	27
4	Performance Evaluation	27
4.1	Performance Analyses	27
4.2	Performance Test	29
4.2.1	Detailed Configuration of Implementation Platforms	29
4.2.2	Performance Test Results	30
5	Feature Statements	31
5.1	Innovativeness	31

5.1.1 Unified Decoupling Framework	31
5.1.2 Counter-Based Domain Multiplexing	31
5.1.3 Mandatory Low-Latency Fast Path	31
5.2 Simplicity	31
5.2.1 Counter-Derived Round Functions	31
5.2.2 Minimalist IV Design	32
5.2.3 Unified Construction	32
5.3 Flexibility	32
5.3.1 Parameterizable Permutation Support	32
5.3.2 Extensible Cryptographic Primitives	32
5.3.3 Dedicated Mechanism for Short Messages	32
5.4 Performance Advantages	33
5.4.1 Superior Hash Effectiveness	33
5.4.2 Nibble-Sliced Bit-Parallel Architecture	33
A Implementation Notes and Clarifications	36
B The Lane-Dedicated eXtendable Output Function for Short Messages	37
C Security Proofs for Rocket-JH	39
C.1 Security Definitions	39
C.2 CTR-Perm and CTR-Func	41
C.3 Decohash Framework	42
C.3.1 Generic Composition	42
C.3.2 Decohash on CTR-Perm	44
C.3.3 Decohash on CTR-Func	45
C.4 Indifferentiability of Rocket-JH on CTR-Perm	47
C.4.1 R1 on Ideal Random Permutations	47
C.4.2 R1 on CTR-Perm	51
C.4.3 Indifferentiability of Rocket-1f	52
C.5 Indifferentiability of Rocket-JH on CTR-Func	52
C.5.1 R1 on CTR-Func: Analysis without the Parameter t	52
C.5.2 Indifferentiability of Rocket-1f	57
C.6 Turning Indifferentiability to Hash Security	59
C.7 Direct Proofs for the Preimage of Rocket-JH on CTR-Perm	60
C.7.1 Direct Preimage Analysis	60
C.7.2 Direct Second-Preimage Analysis	61
C.8 Direct Proofs for the Preimage of Rocket-JH on CTR-Func	62
C.8.1 Direct Preimage Analysis	62
C.8.2 Direct Second-Preimage Analysis	63
C.9 Indifferentiability of LDXOF and LDRH	64
D The $\mathcal{M}$ -Layer Matrix	66
E Statistical Test Results of C Hash	67
E.1 Randomness Statistical Test Results of C-Hash-512	67
E.2 Randomness Statistical Test Results of C-Hash-1024	67

List of Figures

1	Modern Artistic Interpretation of the Chitù Warhorse	5
2	Rocket-JH: Three-Phase Construction Overview	7
3	CTR-Perm and CTR-Func Based on a Permutation P	8
4	One Round of the Extended Lai-Massey Structure	18
5	Decohash Framework: VIL-Hash and FIL-Finalization Architecture	42

List of Tables

1	Counter Payload Allocation for All C Hash Modes	9
2	Byte-to-Nibble Mapping	11
3	The C-Engine 4-bit S-box	13
4	The 28 Round Constants of C-Engine	13
5	C Hash Family Parameter Specifications	14
6	Security Summary for C Hash Family	22
7	Software Implementation Platform Configuration	29
8	Throughput by Instance and Message Length	30
9	Computational Efficiency by Instance and Message Length	30
10	Software Resource Consumption	30
11	Randomness Statistical Test Results of C-Hash-512	67
12	Randomness Statistical Test Results of C-Hash-1024	67

1 Algorithm Description

1.1 Overview

Naming and Philosophy. The C Hash family takes its name from Chìtù (Pinyin: chìtù, pronounced *chee-too*), the legendary warhorse of the Eastern Han dynasty, universally celebrated as the epitome of equine excellence in Chinese history, as depicted in Fig. 1. Just as its namesake embodies the dual virtues of formidable strength and exceptional agility on the battlefield, C Hash delivers both *cryptographic robustness* and *implementation efficiency* in modern secure communication systems.

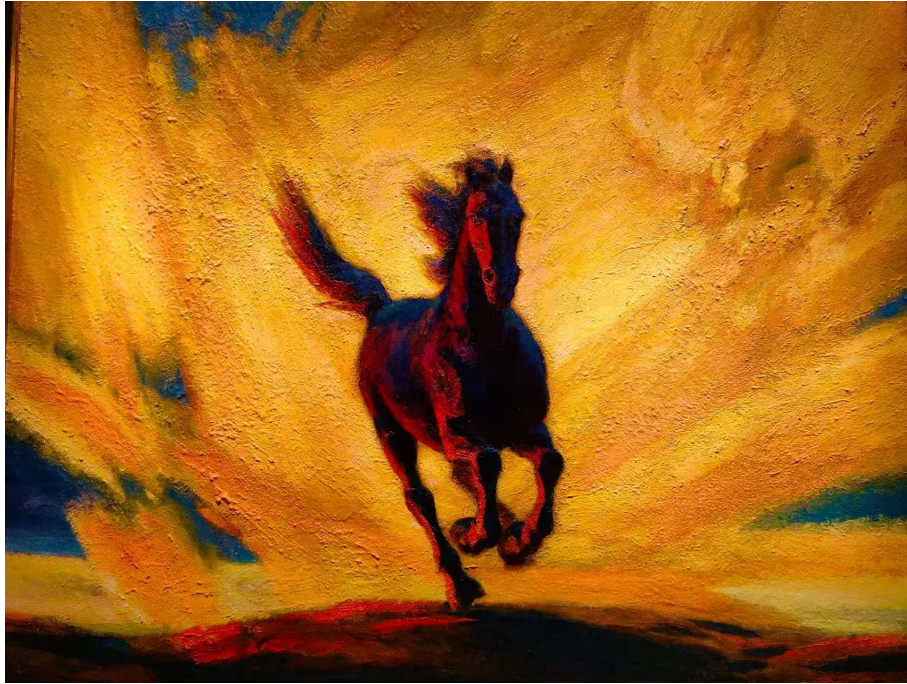


Fig. 1. Modern artistic interpretation of the Chìtù warhorse by artist Jianjun Zhan, exhibited at the National Art Museum of China (photograph taken during the 2026 Spring Festival).

Design Objectives. Within the provable security paradigm, C Hash targets *high indistinguishability* and *high efficiency* from an ideal random oracle. It achieves 512-bit security bounds against distinguishing attacks and a 736-bit message absorption rate while using only a 1536-bit permutation core, **C-Engine**—a remarkable security-efficiency trade-off.

Mode of Operation. The construction adopts the **Rocket-JH** iteration [22] atop the 1536-bit permutation C-Engine. It uses a unified domain-separation mechanism that derives independent function instances via a counter **ctr**:

- **CTR-Perm:** Direct permutation-based processing, offering optimal efficiency for high-throughput scenarios;
- **CTR-Func:** Strengthened variant with feedforward XOR on CTR-Perm, to disable backward queries and achieve enhanced preimage security bounds.

Family Instances. We define two instantiations distinguished by their respective modes of operation:

C-Hash-512 Employs **CTR-Perm** for performance-critical applications, delivering 512-bit outputs with minimal latency;

C-Hash-1024 Employs **CTR-Func** for heightened security requirements, delivering 1024-bit outputs with superior resistance to preimage and second-preimage attacks.

Both instances automatically invoke the Lane-Dedicated Rapid Hash (**LDRH**) for short messages ($|M| < n$), bypassing the standard VIL+FIL iteration.

Execution Flow. For any message M with bit length $|M| < n \cdot 2^{w-8}$, C Hash proceeds as follows:

1. **Short-Message Check:** If $|M| < n$, C Hash invokes LDRH and returns.
2. **Initialization:** Derives IV_{hashlen} and calculates $h_{0,1}, h_{0,2}$ via CTR-Perm/CTR-Func.
3. **VIL Compression:** Through the Rocket-JH iteration, absorbs padded message blocks $m_1, \dots, m_l$ using pairwise distinct CTR-Perm/CTR-Func instances.
4. **FIL Finalization:** Extracts z output blocks via distinct CTR-Perm/CTR-Func instances and truncates to hashlen bits.

1.2 The Rocket-JH Iteration

1.2.1 Symbols This document uses the following symbols.

0^k	: the k -bit all-zero string, i.e., k consecutive zero bits
$\{0, 1\}^*$	: the set of all finite-length binary strings
$\{0, 1\}^n$	: the set of n -bit strings
$ x $	: the bit-length of string x
$[x]_n$	: n -bit binary encoding of string or integer x
$x \parallel y$	: concatenation of strings x and y
$x \oplus y$	: bitwise XOR operation (defined only when $ x = y $)
h	: output block size extracted per finalization permutation
hashlen	: target hash output length in bits
$\text{LMB}(a, x)$	: extract the leftmost a bits of x (for $a \leq x $), i.e., the a most-significant bits

Byte order. All multi-byte integers are encoded in big-endian (network) order; see Appendix A for details.

1.2.2 Message Padding The padding scheme ensures unique message decoding and aligns input with the block processing granularity. Specifically, every message $M \in \{0, 1\}^*$ undergoes the following transformation:

1. **Bit Appending:** Append a single ‘1’ bit to M , marking the message boundary.
2. **Zero Padding:** Append the minimum number of ‘0’ bits (possibly none) such that the total length becomes a multiple of the block size $n/2 = 736$ bits.

Formally, the padding function $\mathbf{Pad} : \{0, 1\}^* \rightarrow \{0, 1\}^{736 \cdot l}$ produces:

$$\mathbf{Pad}(M) = M \parallel 1 \parallel 0^k, \quad \text{where } k = \min\{j \geq 0 : |M| + 1 + j \equiv 0 \pmod{736}\}$$

We denote this operation by $m_1 m_2 \dots m_l \leftarrow \mathbf{Pad}(M)$, where the padded result is partitioned into l consecutive 736-bit blocks $m_i \in \{0, 1\}^{736}$ for $i \in [1, l]$.

This padding method satisfies three essential properties: *unambiguous decoding* (the trailing ‘1’ uniquely identifies the original message end), *minimal overhead* (at most 735 padding bits), and *alignment efficiency* (736-bit blocks match the state variable size $n/2$, enabling streamlined implementation).

In byte-oriented implementations, the appended 1 bit is placed immediately after the last message bit within the current byte, with subsequent bits zeroed out; see Appendix A for the bit-level procedure.

1.2.3 The Rocket-JH Construction Following the *Compress-then-Randomize* paradigm [22], Rocket-JH architecturally decouples the hash function into a variable-input-length (VIL) compression component and a fixed-input-length (FIL) finalization component. This structural separation enables the VIL phase to maximize message absorption throughput while the FIL phase provides indistinguishability from a random oracle, effectively breaking the conventional rate-capacity trade-off inherent in homogeneous designs like sponge or Merkle-Damgård.

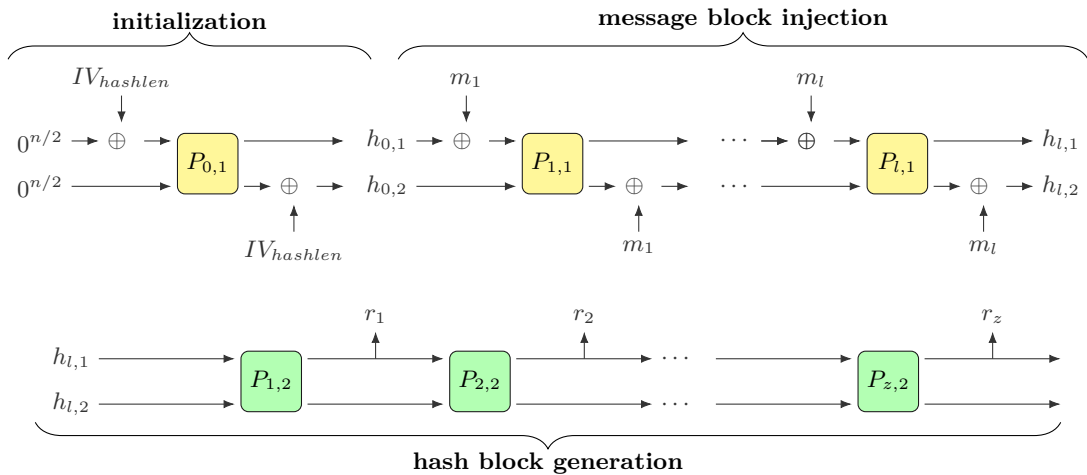


Fig. 2. Rocket-JH: Three-Phase Construction Overview, with $ID_{vil} = 1$ for VIL compression and $ID_{fil} = 2$ for FIL finalization. Here the subscripts 1 and 2 are schematic shorthand for the concrete LaneIDs $0x01$ and $0x02$ listed in Table 5.

Building upon the JH iteration [21], the construction introduces a distinctive feature: the use of *pairwise distinct round permutations* $P_{i, \text{ID}_{\text{vil}}}$ ($i \in [0, l]$) for VIL compression and $P_{i, \text{ID}_{\text{fil}}}$ ($i \in [1, z]$) for FIL finalization, all operating on $\{0, 1\}^n$. The design is illustrated in Fig. 2 and detailed in Alg. 1.

Algorithm 1: ROCKET-JH SPECIFICATION

Input: Message $M \in \{0, 1\}^*$, hash value length hashlen , output block length h
Output: Digest hash

// Initialization Phase

1 $h_{0,1} \parallel h_{0,2} \leftarrow P_{0, \text{ID}_{\text{vil}}} (IV_{\text{hashlen}} \parallel 0^{n/2}) \oplus (0^{n/2} \parallel IV_{\text{hashlen}})$, with $|IV_{\text{hashlen}}| = n/2$;

// Message Block Injection Phase

2 $m_1 m_2 \cdots m_l \leftarrow \text{Pad}(M)$, with $|m_i| = n/2$ for $i \in [1, l]$;

3 **for** $i \leftarrow 1$ **to** l **do**

4 $x_{i,1} \leftarrow h_{i-1,1} \oplus m_i$, $x_{i,2} \leftarrow h_{i-1,2}$;

5 $y_{i,1} \parallel y_{i,2} \leftarrow P_{i, \text{ID}_{\text{vil}}} (x_{i,1} \parallel x_{i,2})$, with $|y_{i,1}| = |y_{i,2}| = n/2$;

6 $h_{i,1} \leftarrow y_{i,1}$, $h_{i,2} \leftarrow y_{i,2} \oplus m_i$;

// Hash Block Generation Phase

7 $z \leftarrow \lceil \text{hashlen}/h \rceil$;

8 **for** $i \leftarrow 1$ **to** z **do**

9 $h_{l+i,1} \parallel h_{l+i,2} \leftarrow P_{i, \text{ID}_{\text{fil}}} (h_{l+i-1,1} \parallel h_{l+i-1,2})$, with $|h_{l+i,1}| = h$ and $|h_{l+i,2}| = n - h$;
 ; // When $h = n$, the second component $h_{l+i,2}$ is the empty string ε

10 $r_i \leftarrow h_{l+i,1}$;

11 $\text{hash} \leftarrow \text{LMB}(\text{hashlen}, r_1 \parallel r_2 \parallel \cdots \parallel r_z)$;

12 **return** hash .

1.2.4 Domain Separation: CTR-Perm and CTR-Func To realize the pairwise distinct round functions required by Rocket-JH in a practical and efficient manner, we introduce CTR-Perm and CTR-Func, a unified domain-separation framework that derives 2^w effectively independent function instances from a single large permutation.

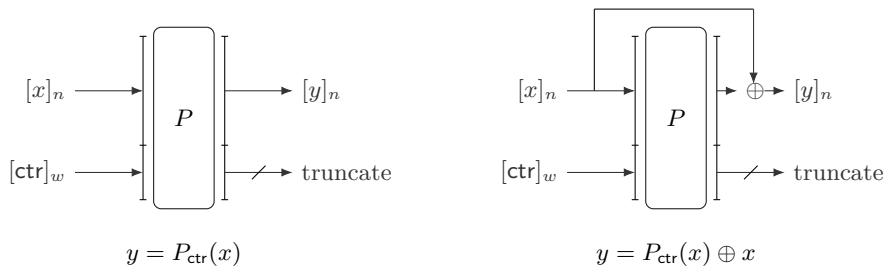


Fig. 3. CTR-Perm (left) and CTR-Func (right) based on a permutation P . The counter ctr occupies the rightmost w bits and x the leftmost n bits of the b -bit input.

As illustrated in Fig. 3, the $b = n + w$ input bits are partitioned into an n -bit message block x and a w -bit counter ctr . The permutation is evaluated on the concatenation $x \parallel \text{ctr}$, but only the leftmost n bits (i.e., the most-significant n bits) of the output are retained; the remaining w bits are discarded.

Formally, for a random permutation P and a fixed counter $\text{ctr} \in \{0,1\}^w$, we define:

$$\text{CTR-Perm: } P_{\text{ctr}}(x) = \text{LMB}(n, P(x \parallel \text{ctr})), \quad (1)$$

$$\text{CTR-Func: } P'_{\text{ctr}}(x) = \text{LMB}(n, P(x \parallel \text{ctr})) \oplus x. \quad (2)$$

The feedforward XOR in CTR-Func ensures that inverting P'_{ctr} requires solving $y = P_{\text{ctr}}(x) \oplus x$ for x given y , which is computationally infeasible without knowledge of P 's internal structure. Consequently, backward queries are effectively disabled: the success probability of any inversion attempt is bounded by $2^{-(b-w)}$.

Unified Counter Layout. All CTR-Perm and CTR-Func invocations share a common 64-bit counter format:

$$\text{ctr} = [\text{LaneID}]_8 \parallel [\text{Payload}]_{56},$$

where $[\text{LaneID}]_8$ occupies bits 63–56 (most-significant byte) and $[\text{Payload}]_{56}$ occupies bits 55–0. Table 1 details the exact payload allocation for each C Hash operational mode.

Table 1. Counter Payload Allocation for All C Hash Modes

Mode	LaneID	Payload Field	Width	Content / Range
Rocket-JH VIL	0x01 / 0x81	Round Index i	56 bits	$i \in [0, 2^{56})$
Rocket-JH FIL	0x02 / 0x82	Round Index i	56 bits	$i \in [1, z]$
LDRH	0x03 / 0x83	Instance ID	56 bits	Values 1 or 2
LDXOF	0x04 / 0x84	OutputByteLen	24 bits	$[0, 2^{24})$ bytes
		Reserved	8 bits	Fixed to 0x00
		Instance i	24 bits	$i \in [0, l]$

Placing LaneID in the most-significant byte guarantees that carry propagation from incrementing the lower 56 bits can never alter the algorithm identity, preserving strict cryptographic isolation across all lanes.

The choice between CTR-Perm and CTR-Func enables flexible security-efficiency trade-offs: CTR-Perm offers optimal efficiency for environments where backward queries are feasible, while CTR-Func provides hardened security for adversarial settings requiring full preimage resistance.

1.2.5 The Lane-Dedicated Rapid Hash (LDRH) To enable rapid hashing of short messages without incurring the two-phase overhead (VIL absorption + FIL finalization) of the standard Rocket-JH constructions, we introduce a *dedicated fast path* within the CTR-Perm framework. This path leverages the construction $P_1(x) \oplus P_2(x)$, where two domain-separated instances of the underlying permutation are invoked in parallel and their outputs combined by XOR, as specified in Alg. 2.

Algorithm 2: THE LANE-DEDICATED RAPID HASH (LDRH)

Input: Message $M \in \{0, 1\}^*$ with $|M| < n$, target length $\text{hashlen} \leq n$, dedicated lane identifier $\text{LaneID} \in \{0, 1\}^8$, Permutation $P : \{0, 1\}^b \rightarrow \{0, 1\}^b$ where $b = n + w$

Output: Digest $\text{hash} \in \{0, 1\}^{\text{hashlen}}$

```

1  $x \leftarrow M \parallel 1 \parallel 0^{n-1-|M|}$ ; // n-bit padded message
2  $\text{ctr}_1 \leftarrow [\text{LaneID} \parallel 1]_w$ ; // counter for first instance
3  $\text{ctr}_2 \leftarrow [\text{LaneID} \parallel 2]_w$ ; // counter for second instance
4  $y_1 \leftarrow \text{LMB}(n, P(x \parallel \text{ctr}_1))$ ; // first CTR-Perm call
5  $y_2 \leftarrow \text{LMB}(n, P(x \parallel \text{ctr}_2))$ ; // second CTR-Perm call
6  $y \leftarrow y_1 \oplus y_2$ ; // XOR of two independent outputs
7  $\text{hash} \leftarrow \text{LMB}(\text{hashlen}, y)$ ;
8 return  $\text{hash}$ .
```

Mandatory Short-Message Routing. For $|M| < n$, both C-Hash-512 and C-Hash-1024 must invoke the LDRH lane (LaneID 0x03 and 0x83, respectively) instead of the standard VIL+FIL iteration.

Recall that the domain-separation framework employs an 8-bit LaneID field to derive independent function instances. We designate LaneID = 0x03 for C-Hash-512 and LaneID = 0x83 for C-Hash-1024 as the dedicated *Short-Message Rapid Hash* channels, reserved exclusively for LDRH and never used by the standard multi-block lanes of the C Hash family. The two parallel instances are further separated by the least-significant counter bits 0x01 and 0x02, ensuring that P_1 and P_2 are effectively independent random functions under the CTR-Perm model.

The XOR of two domain-separated permutations has been shown to achieve full indifferentiable security from a random function, with tight bounds established via the χ^2 method [4], modular proof frameworks [7], improved simulator strategies [13], and Fourier analysis [10]. Collectively, these works confirm that $P_1(x) \oplus P_2(x)$ effectively destroys the algebraic structure (fixed points, invertibility, and cyclic behavior) inherent to a single permutation, yielding an ideal fixed-input-length random function.

Consequently, LDRH is tailored for prevalent latency-sensitive workloads — such as packet authentication, handshake transcripts, and commitment schemes — where short inputs dominate.

Adopting the same XOR-sum design principle, Appendix B presents the Lane-Dedicated eXtendable Output Function (LDXOF), which extends this approach to produce arbitrarily long outputs from short seeds for public-key primitives such as seed-expansion in digital signatures and commitment schemes.

1.3 The C-Engine Permutation

1.3.1 Algorithm Overview The permutation $P : \{0, 1\}^{1536} \rightarrow \{0, 1\}^{1536}$, referred to as C-Engine, constitutes the sole cryptographic core of both C Hash instances. It operates on a 1536-bit input X by first parsing it into an internal state matrix, then applying 28 rounds of an iterated round function, and finally serializing the state back into a 1536-bit output Y .

State Encoding. The 1536-bit internal state is organized as a 6×64 matrix of nibbles:

$$\mathbf{S} = (s_{i,j}), \quad i \in [0, 5], \quad j \in [0, 63],$$

where each $s_{i,j} \in \{0, 1\}^4$. Thus the state comprises $6 \times 64 = 384$ nibbles, i.e., 1536 bits in total. The matrix is filled with the 192-byte input $X = X_0 X_1 \dots X_{191}$ in row-major order: row i contains the contiguous 32-byte segment $X_{32i} \dots X_{32i+31}$. Within each byte, the high nibble (leftmost 4-bit) is mapped to the even column index $2k$, and the low nibble (rightmost 4-bit) to the immediately following odd column index $2k + 1$, as summarized in Table 2 and Eq. (3).

Table 2. Byte-to-Nibble Mapping for Row i (Columns 0–63)

Byte index	$32i$		$32i + 1$		$32i + 2$		$\dots$	$32i + 30$		$32i + 31$	
Nibble position	hi	lo	hi	lo	hi	lo	$\dots$	hi	lo	hi	lo
Matrix column j	0	1	2	3	4	5	$\dots$	60	61	62	63
Matrix entry	$s_{i,0}$	$s_{i,1}$	$s_{i,2}$	$s_{i,3}$	$s_{i,4}$	$s_{i,5}$	$\dots$	$s_{i,60}$	$s_{i,61}$	$s_{i,62}$	$s_{i,63}$

$$\mathbf{S} = \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} & \dots & s_{0,62} & s_{0,63} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} & \dots & s_{1,62} & s_{1,63} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ s_{5,0} & s_{5,1} & s_{5,2} & s_{5,3} & \dots & s_{5,62} & s_{5,63} \end{pmatrix} \begin{array}{l} \leftarrow \text{bytes } X_0 \dots X_{31} \\ \leftarrow \text{bytes } X_{32} \dots X_{63} \\ \vdots \\ \leftarrow \text{bytes } X_{160} \dots X_{191} \end{array} \quad (3)$$

Formally, the parsing function PARSE TOMATRIX is defined as follows. For row $i \in [0, 5]$ and column $j \in [0, 63]$, let $k = \lfloor j/2 \rfloor$ be the byte offset within the row. Then

$$s_{i,j} = \begin{cases} \lfloor X_{32i+k}/16 \rfloor, & j \bmod 2 = 0 \text{ (high nibble),} \\ X_{32i+k} \bmod 16, & j \bmod 2 = 1 \text{ (low nibble).} \end{cases}$$

The inverse mapping SERIALIZE reassembles each byte from two constituent nibbles. For byte index $b \in [0, 191]$, let $i = \lfloor b/32 \rfloor$ and $k = b \bmod 32$; then

$$X_b = s_{i,2k} \parallel s_{i,2k+1},$$

where $s_{i,2k}, s_{i,2k+1} \in \{0, 1\}^4$ are the high and low nibbles of X_b , respectively, and $\parallel$ denotes concatenation.

Round Function. C-Engine processes the parsed state $\mathbf{S}$ through 28 rounds. Each round $r \in [0, 27]$ applies three transformations in sequence:

1. **S-Layer (Non-linear Confusion):** 64 parallel 24-bit S-boxes, each operating on one column of 6 nibbles $(s_{0,j}, \dots, s_{5,j})$. The 24-bit S-box is constructed by 4-bit small S-box using SPS structure.
2. **M-Layer (Linear Diffusion):** A 64×64 binary matrix M_{64} over $\text{GF}(2^4)$ applied independently to each row of 64 nibbles.

3. **$\mathcal{K}$ -Layer (Round Constant):** The 32-bit round constant $RC^{(r)}$ is XORed with the first 8 nibbles (4 bytes) of Row 0; the remaining 56 nibbles of Row 0 and all other rows are left unchanged.

Formally, the round function is defined as

$$\text{Round}_r(\mathbf{S}) = \mathcal{K}_r \circ \mathcal{M} \circ \mathcal{S}(\mathbf{S}),$$

and the full permutation is the composition of 28 such rounds:

$$\text{C-Engine}(X) = \text{Round}_{27} \circ \dots \circ \text{Round}_0(\text{ParseToMatrix}(X)).$$

Algorithm 3 gives the complete specification. The outer loop enumerates the 28 rounds. Inside each round, the first inner loop iterates over the 64 columns to apply the $\mathcal{S}$ -Layer; the second inner loop iterates over the 6 rows to apply the $\mathcal{M}$ -Layer; finally the $\mathcal{K}$ -Layer injects the round constant into Row 0. After all rounds, the state is serialized back to the 192-byte output Y .

Algorithm 3: C-ENGINE: THE 1536-BIT PERMUTATION

Input: $X \in \{0, 1\}^{1536}$
Output: $Y \in \{0, 1\}^{1536}$

```

1  $\mathbf{S} \leftarrow \text{PARSETOMATRIX}(X);$  // Map 192-byte input to  $6 \times 64$  nibble matrix
2 ;
3 for  $r = 0$  to 27 do
    //  $\mathcal{S}$ -Layer: 64 parallel 24-bit S-boxes
4     for  $j = 0$  to 63 do
5          $(a_0, \dots, a_5) \leftarrow (s_{0,j}, \dots, s_{5,j});$ 
6         for  $i = 0$  to 5 do
7              $b_i \leftarrow S(a_i);$  // 4-bit small sbox
8          $(c_0, \dots, c_5) \leftarrow M_{6 \times 6} \cdot (b_0, \dots, b_5)^\top;$  //  $M_{6 \times 6}$  over  $\text{GF}(2^4)$ 
9          $(c_0, \dots, c_5) \leftarrow (c_0, \dots, c_5) \oplus (0\mathbf{x}C, 0\mathbf{x}9, 0\mathbf{x}0, 0\mathbf{x}8, 0\mathbf{x}B, 0\mathbf{x}2);$  // 24-bit constant
            //  $0\mathbf{x}C908B2$  from  $\sqrt{2}$  in big-endian nibble order
10        for  $i = 0$  to 5 do
11             $s_{i,j} \leftarrow S(c_i);$  // 4-bit small sbox

    //  $\mathcal{M}$ -Layer: row-wise diffusion
12    for  $i = 0$  to 5 do
13         $(s_{i,0}, \dots, s_{i,63}) \leftarrow M_{64} \cdot (s_{i,0}, \dots, s_{i,63})^\top;$  // The matrix  $M_{64}$  is a  $64 \times 64$  binary matrix

    //  $\mathcal{K}$ -Layer: round constant injection into Row 0
14     $(s_{0,0}, \dots, s_{0,7}) \leftarrow (s_{0,0}, \dots, s_{0,7}) \oplus \text{EIGHTNIBBLES}(RC^{(r)});$  // Converts 32-bit  $RC^{(r)}$  to 8
        nibbles, high nibble first

15  $Y \leftarrow \text{SERIALIZE}(\mathbf{S});$  // Map  $6 \times 64$  nibble matrix back to 192-byte output
16 ;
17 return  $Y$ .
```

1.3.2 Component Specifications

The $\mathcal{S}$ -Layer. Each of the 64 columns is transformed by a 24-bit S-box, which is constructed by 4-bit small S-box using SPS structure: a substitution layer of 4-bit S-boxes (Table 3), a lightweight binary matrix $M_{6 \times 6}$ over $GF(2^4)$ (Equation (4)), an XOR with the 24-bit internal constant $0xC908B2$ (derived from a 24-bit segment of the binary expansion of $\sqrt{2}$), and then a second substitution layer of 4-bit S-boxes.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$S(x)$	1	4	0	c	3	2	5	b	a	8	6	f	7	9	d	e

Table 3. The C-Engine 4-bit S-box

$$M_{6 \times 6} = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 1 \end{pmatrix}$$

Equation (4): Diffusion matrix over $GF(2^4)$

The $\mathcal{M}$ -Layer. A 64×64 binary matrix M_{64} over $GF(2^4)$ is applied independently to each row. This matrix is constructed via a 4-round extended Lai-Massey structure, which can achieve high branch number. The detailed method is specified in the design rationale section, and the contents of the binary matrix M_{64} are specified in Appendix D. For completeness, each round of the structure is defined as follows:

$$S \leftarrow P_1(L \oplus R), \quad L' \leftarrow L \oplus S, \quad R' \leftarrow P_2(R \oplus S),$$

where P_1 is a round-dependent cyclic left shift ($\lll$) with nibble-shift amounts (15, 7, 23, 16) for rounds 1–4 respectively and P_2 is a fixed cyclic right shift by 1 nibble ($\ggg 1$).

The $\mathcal{K}$ -Layer. Twenty-eight 32-bit round constants $RC^{(0)}, \dots, RC^{(27)}$ are derived from the fractional part of π . In round r , $RC^{(r)}$ is XORed with the first 8 nibbles (4 bytes) of Row 0. The constant $RC^{(0)} = 0x243F6A88$ corresponds to the byte sequence (0x24, 0x3F, 0x6A, 0x88), and is applied to Row 0 as the nibbles $\text{EIGHTNIBBLES}(RC^{(0)}) = (0x2, 0x4, 0x3, 0xF, 0x6, 0xA, 0x8, 0x8)$. The remaining 56 nibbles of Row 0 and all other rows are left unchanged. The constants are listed in Table 4.

Table 4. The 28 round constants of C-Engine (32-bit, hexadecimal)

r	$RC^{(r)}$	r	$RC^{(r)}$	r	$RC^{(r)}$	r	$RC^{(r)}$	r	$RC^{(r)}$	r	$RC^{(r)}$	r	$RC^{(r)}$
0	0x243F6A88	1	0x85A308D3	2	0x13198A2E	3	0x03707344	4	0xA4093822	5	0x299F31D0	6	0x082EFA98
7	0xEC4E6C89	8	0x452821E6	9	0x38D01377	10	0xBE5466CF	11	0x34E90C6C	12	0xC0AC29B7	13	0xC97C50DD
14	0x3F84D5B5	15	0xB5470917	16	0x9216D5D9	17	0x8979FB1B	18	0xD1310BA6	19	0x98DFB5AC	20	0x2FFD72DB
21	0xD01ADFB7	22	0xB8E1AFED	23	0x6A267E96	24	0xBA7C9045	25	0xF12C7F99	26	0x24A19947	27	0xB3916CF7

1.4 C Hash Parameters

The C Hash family comprises two algorithm instances, C-Hash-512 and C-Hash-1024, sharing a unified architectural framework with mode-specific optimizations. Both instances are parameterized as follows:

- **Permutation size** (b): The bit-width of the underlying permutation P .
- **Counter size** (w): The bit-width allocated for domain separation counters.
- **Effective block size** (n): Defined as $n = b - w$, determining the state capacity for cryptographic operations.
- **Initial value** (IV_{hashlen}): Derived directly from `hashlen`, encoded as an $n/2$ -bit binary string $[\text{hashlen}]_{n/2}$.
- **Message block size** ($n/2$): The granularity of message absorption per round.
- **Hash length** (`hashlen`): The desired bit-length of the output digest.
- **Output block size** (h): The bit-length extracted per finalization permutation.
- **Number of output blocks** ($z = \lceil \text{hashlen}/h \rceil$): The count of finalization rounds required to produce the full digest.
- **Permutation mode**: CTR-Perm for optimized efficiency, or CTR-Func for hardened backward-query resistance.
- **Lane identifiers** (`LaneID`): The 8-bit lane index selects among cryptographically isolated permutation instances. Each algorithm instance employs a triple $(ID_{\text{vil}}, ID_{\text{fil}}, ID_{\text{ldrh}})$: ID_{vil} governs IV initialization and message block injection (VIL compression), ID_{fil} governs hash block generation (FIL finalization), and ID_{ldrh} enables LDRH for short messages.
- **Lane-Dedicated Rapid Hash (LDRH)**: A fast path for short messages ($|M| < n$ bits) that bypasses the iterative VIL absorption and FIL finalization phases. LDRH is mandatory and uses the dedicated `LaneID` listed below; it always employs CTR-Perm and never CTR-Func.

Table 5 summarizes the concrete parameter instantiations.

Table 5. C Hash Family Parameter Specifications

Name	hashlen	IV	(b, w, n)	(z, h)	Mode	$(ID_{\text{vil}}, ID_{\text{fil}}, ID_{\text{ldrh}})$	LDRH
C-Hash-512	512	0x00...0200	(1536, 64, 1472)	(1, 736)	CTR-Perm	(0x01, 0x02, 0x03)	mandatory
C-Hash-1024	1024	0x00...0400	(1536, 64, 1472)	(1, 1472)	CTR-Func	(0x81, 0x82, 0x83)	mandatory

The IV values denote the target hash length in hexadecimal shorthand; the complete IV_{hashlen} is the 736-bit binary encoding $[\text{hashlen}]_{736}$ with leading zeros. The 8-bit `LaneID` employs the most significant bit in the counter as an *instance selector*: values in the range 0x01–0x03 are reserved for C-Hash-512, while 0x81–0x83 are reserved for C-Hash-1024. This structural separation guarantees cryptographically isolated domains across algorithm instances, preventing cross-mode query cycles in the CTR-Perm/CTR-Func framework.

2 Design Rationale

2.1 The Rationales of the Rocket-JH Mode

2.1.1 The Homogeneity Barrier

Security: The Foundational Pillar. Security stands as the paramount consideration. For hash functions, three classical properties form the baseline: *collision resistance*, *preimage resistance*, and *second-preimage resistance*. However, modern applications demand a stronger guarantee: *randomness*—the behavior must be computationally indistinguishable from a random oracle [2]. This requirement underpins critical primitives including digital signatures, MACs, KDFs, and commitment schemes. The *indifferentiability* framework [17,8] provides the rigorous foundation for assessing this property.

The Efficiency-Cost Dilemma. At the construction level, the *rate*—the message block length processed per primitive invocation—predominantly determines *efficiency*. Conversely, the *primitive size* (the bit width b of the underlying permutation) governs *cost*. The ideal design maximizes rate while minimizing b .

Yet conventional homogeneous constructions face an inherent tension:

- **Merkle-Damgård** [18,9]: High rate, but length-extension vulnerabilities.
- **sponge** [3]: Simple structure, but under fixed width b , security degrades as the rate r grows because capacity $c = b - r$ shrinks.
- **JH** [21]: Wide state, but homogeneous recycling of the same permutation limits indifferentiability to $b/4$ bits [19].

In all cases, improving one dimension (security, efficiency, or cost) necessarily degrades another because the *same* primitive is forced to serve conflicting objectives simultaneously. Cross-round query cycles and input collisions with the identical round function entangle the probability analysis across rounds, making it difficult to obtain tight composable bounds. This accumulated proof complexity, in turn, forces conservative parameter choices that further entrench the antagonism between security and efficiency.

2.1.2 Structural Decoupling: The Compress-then-Randomize Paradigm

These limitations of monolithic constructions necessitate a paradigm shift: rather than forcing a single primitive to satisfy conflicting objectives, we *architecturally decouple* the hash function into two specialized components. We formalize this approach as the *Decohash* framework, which composes a variable-input-length (VIL) compression function H with a fixed-input-length (FIL) finalization function F ; see Fig. 5 (Appendix C.3.1) for the abstract architecture and Appendix C.3.2 and Appendix C.3.3 for the full indifferentiability analysis.

1. **Variable-Input-Length (VIL) Compression:** Focusing solely on *collision resistance* and *preimage resistance*, liberating it to maximize processing rate without the burden of full randomness.
2. **Fixed-Input-Length (FIL) Finalization:** Employing independent permutations to provide *strong randomness extraction*, ensuring indifferentiability on a short, fixed input.

This separation breaks the conventional rate-capacity trade-off: the VIL phase optimizes for speed while the FIL phase independently guarantees random-oracle-like behavior.

2.1.3 Temporal Decoupling: Pairwise Distinct Round Functions

Structural decoupling separates the hash function into two spatial components (VIL vs. FIL); temporal decoupling extends this principle to the iterative dimension itself.

Unlike homogeneous designs (e.g., sponge) that recycle a single primitive, we mandate *pairwise distinct round functions* across all processing rounds. This temporal decoupling eliminates cross-round query cycles and algebraic dependencies.

The advantages are immediate:

- **Clean Security Analysis:** Each round operates as an independent entity; bounds factorize without accounting for inter-round correlations.
- **Linear Scaling:** Collision probabilities obey the birthday bound per round, yielding $O(q^2l/2^n)$ bounds rather than scaling quadratically in message length.
- **Strict Isolation:** VIL vulnerabilities cannot propagate to compromise FIL finalization.

2.1.4 From JH to Rocket-JH: CTR-Perm and CTR-Func

Why JH? We select the JH iteration as the VIL backbone because its b -bit permutation already operates on a wide state with a natural $b/2$ message injection rate. Unlike narrow-pipe Merkle-Damgård, JH provides ample internal state to accommodate both message absorption and capacity without artificial rate-capacity tension. This makes it an ideal substrate for decoupling: the VIL phase can absorb at the full $n/2 = 736$ -bit rate while leaving sufficient state entropy for the FIL phase to extract strong randomness.

Domain-Separated Multiplexing. While decoupling delivers superior security bounds, it imposes a severe implementation constraint: *pairwise distinct round functions* require $l + z + 1$ independent cryptographic primitives, which is prohibitively expensive in hardware. CTR-Perm and CTR-Func resolve this practical barrier through *domain-separated multiplexing*—deriving 2^w effectively independent function instances from a single large permutation core $P : \{0, 1\}^b \rightarrow \{0, 1\}^b$ via counter-based input diversification.

CTR-Perm ($P_{\text{ctr}}(x) = \text{LMB}(n, P(x \parallel \text{ctr}))$) instantiates C-Hash-512 for performance-critical scenarios. For heightened security, CTR-Func augments this with a feedforward XOR ($P'_{\text{ctr}}(x) = P_{\text{ctr}}(x) \oplus x$), disabling backward queries thereby instantiating C-Hash-1024 with 1024-bit preimage resistance. Together, they achieve *implementation-level decoupling*: a single cryptographic core delivers the analytical benefits of distinct permutations while maintaining cost efficiency.

2.1.5 Latency-Aware Decoupling: The LDRH Fast Path

Standard two-phase hashing (VIL absorption + FIL finalization) incurs fixed overhead—initialization, padding alignment, and at least one permutation call—that dominates latency when $|M| \ll n$. For prevalent workloads such as

TLS handshake transcripts, packet authentication tags, and commitment schemes, inputs are typically well below 1472 bits. LDRH eliminates this structural penalty by bypassing the iterative JH chain entirely, using two parallel domain-separated CTR-Perm calls with XOR-combination. This is not merely an optimization; it is a *mandatory* realization of the decoupling principle at the extreme end of the message-length spectrum, ensuring that C Hash delivers uniformly low latency across all input sizes.

2.2 The Rationales of C-Engine

2.2.1 The Wide-Permutation Dilemma The Decohash framework of Rocket-JH imposes two hard constraints on the underlying permutation. First, the effective block size plus the counter field demands a width $b = n + w \geq 1430$ bits (concretely $b = 1536$), far exceeding the block sizes of conventional ciphers. Second, the permutation must be *public* and efficiently multiplexed: CTR-Perm and CTR-Func evaluate the same core P under 2^w distinct counter suffixes, so P must admit fast, branch-free invocation without key-schedule or state-setup overhead.

A direct construction of a 1536-bit “black-box” permutation—for example, via a single large S-box or an random matrix—faces a fundamental dilemma:

- **Cryptanalytic intractability.** Exhaustive characterization of differential, linear, or algebraic properties over a 1536-bit state is computationally impossible. Security assessment would be reduced to heuristic conjecture, lacking the quantitative guarantees required for a standardized primitive.
- **Implementation inefficiency.** A monolithic wide transformation cannot exploit the word-level parallelism of modern processors, resulting in poor throughput and excessive area in hardware.

2.2.2 The Composition Paradigm To resolve this dilemma, C-Engine adopts a *compositional* design philosophy: rather than constructing one inscrutable large primitive, it assembles *small, exhaustively analyzable components* into a hierarchy of increasing width and cryptographic strength. Each step inherits the verifiable security properties of its previous step, and the full 28-round iteration amplifies local diffusion into global pseudorandomness.

This paradigm mirrors the structural decoupling of Decohash at the primitive level: just as Decohash separates VIL compression from FIL finalization, C-Engine decomposes the wide permutation into independent, reusable, and rigorously characterized sub-components.

2.2.3 The Design Rationales of Components The design of C-Engine is grounded in three decades of research on Substitution-Permutation Networks (SPNs) and reflects a proved security cryptographic principles with modern implementation requirements. The following subsections explain the design rationales of the main components decision.

Nibble-Slicing: Analyzable Granularity and Parallelism. The 4-bit *nibble* is chosen as the fundamental processing unit, partitioning the 1536-bit state into 384 independent lanes. Simultaneously, the 4-bit lanes across all 384 nibbles form

contiguous machine words (64-bit, 128-bit, or 256-bit), enabling SIMD-level parallelism through bit-slicing without dedicated vector instructions. The 6×64 nibble matrix thereby maps naturally to both software pipelining on x86/ARM platforms and regular hardware datapaths.

S-Layer: Column-Level — The 24-bit S-box. The **S-Layer** contains a layer of 24-bit S-box in parallel. The 24-bit S-box is constructed by 4-bit small S-box using SPS structure: a substitution layer of 4-bit S-boxes, a lightweight binary matrix $M_{6 \times 6}$ over $\text{GF}(2^4)$, an XOR with the 24-bit internal constant `0xC908B2`, and then a second substitution layer of 4-bit S-boxes. Direct tabulation of a 24-bit S-box would require 2^{24} entries, rendering efficient implementation impractical. C-Engine instead employs a Substitution-Permutation-Substitution (SPS) structure: two layers of 4-bit small S-box separated by a small binary matrix over $\text{GF}(2^4)$.

The 4-bit S-box was selected from the full $16!$ space by balanced multi-objective optimization: zero fixed points, optimal differential uniformity (DDT maximum 4), optimal maximum LAT entry (4), maximum SAC deviation 4, algebraic degree 3, and boomerang connectivity table with $\text{BCT}_{\max} = 6$ and no entries ≥ 8 . The intermediate 6×6 matrix achieves optimal branch number 4 for binary matrix with only simple XOR operations. The resulting 24-bit S-box guarantees strong differential and linear properties. By exhaustive searching, we have obtained the maximal differential probability of the 24-bit S-box is $2^{-8.75}$, and the maximal linear bias of the 24-bit S-box is $2^{-4.09}$. If we use the definition of linear correlation probability as $c = (2\epsilon)^2$, then the maximal linear probability of the 24-bit S-box is $2^{-6.19}$. Moreover, the algebraic degree of the 24-bit S-box achieves 9, which is high enough.

M-Layer: Row-Level — The 64×64 Binary Matrix. The linear part of the round function is a 64×64 binary matrix operating on each row. The binary matrix M_{64} is constructed via a 4-round extended Lai-Massey structure iterated for 4 rounds. In each round, the 64-nibble row is partitioned into left and right halves (L , R , 32-nibble each) and each round consists only nibble-wise XORs and cyclic shifts. The construction structure is illustrated as the following Fig 4.

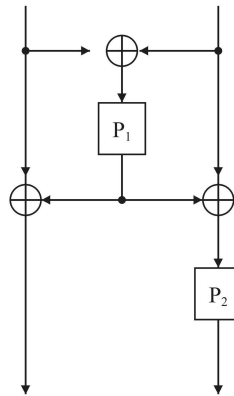


Fig. 4. One Round of the Extended Lai-Massey Structure

The construction of 64×64 binary matrix can be defined as the following operations:

$$S \leftarrow P_1(L \oplus R), \quad L' \leftarrow L \oplus S, \quad R' \leftarrow P_2(R \oplus S),$$

where P_1 is a round-dependent cyclic left shift ($\lll$) with nibble offsets (15, 7, 23, 16) for rounds 1–4 respectively and P_2 is a fixed cyclic right shift by one nibble ($\ggg 1$).

By iterating 4 rounds using the above structure and simple round function, we can get a cryptographically strong binary matrix. Through exhaustive search over rotation parameters, the rotation offsets (15, 7, 23, 16) achieves differential and linear branch number 18 for the 64-dimensional matrix, which achieves the optimal branch number for binary matrix.

Round Function: Permutation-Level — 28-Round Iteration with Independent Constants. The C-Engine permutation utilizes a traditional SP structure, with the nonlinear part is a layer of 24-bit S-box and the linear part is a 64×64 binary matrix with branch number 18. Therefore, we can guarantee that for any two rounds there are at least 18 active S-boxes. Then local diffusion per round is amplified to global confusion by 28 rounds of iteration, which guarantees at least $14 \times 18 = 252$ active 24-bit S-boxes over the full permutation, yielding a maximum differential characteristic probability of $2^{-8.75 \times 252} \approx 2^{-2205}$ and a maximum linear trail correlation of $2^{-6.19 \times 252} \approx 2^{-1560}$. It is obviously both bounds provide a substantial security margin for the C-Engine permutation.

Moreover, to prevent symmetry-based distinguishers, C-Engine permutation employs 28 distinct round constants derived from the fractional part of π , while the S-box internal constant is derived independently from $\sqrt{2}$. This constant separation eliminates algebraic relationships between rounds, mitigating slide attack, rotational attack, invariant subspace attack and other structural attacks.

To sum up, the compositional architecture of C-Engine—from the 4-bit nibble to the 24-bit column S-box, through the 64-nibble row diffusion, and finally to the 28-round iterated permutation—establishes a verifiable chain of security at each granularity. C-Engine permutation simultaneously achieves cryptanalytic tractability, implementation efficiency on modern SIMD architectures, and security margins comparable to an ideal random permutation.

2.3 The Criterion for Parameter Selection

Dual-Profile Design Philosophy. The C Hash family instantiates two profiles from a single framework to cover the full spectrum of deployment scenarios. **C-Hash-512** targets performance-critical environments—high-throughput network appliances, embedded TLS, and real-time streaming—where latency and throughput dominate. CTR-Perm minimizes gate count and cycle count by omitting the feedforward XOR. **C-Hash-1024** targets long-term archival, post-quantum-hardened infrastructure, and high-assurance government systems where preimage resistance must hold even as adversarial resources scale by orders of magnitude. CTR-Func provides this margin at the cost of a single XOR per permutation call, a negligible overhead for the security gained. Both profiles share identical C-Engine silicon and differ only in the counter lane and the feedforward operation, enabling unified hardware with mode-select wiring.

Security-Driven Parameter Bounds. Both C-Hash-512 and C-Hash-1024 are designed for high indistinguishability security. The indistinguishability advantage of Rocket-JH is bounded by:

$$\mathbf{Adv}_{\text{Rocket-JH}}^{\text{indiff}}(q_{\mathcal{S}_i}, q_{\mathcal{R}}) \leq \frac{19q^2l}{2^n} + \frac{(z-1)q^2}{2^n} + \frac{9q^4l}{2^{3n/2+2}} + \frac{O(q^3)}{2^b}.$$

The last term is the chosen multiple-preimage (cm-pre) barrier imposed by the Universal San Mai Attack (USMA) [15]: the adversary manufactures structured target states through the public finalization and amortizes each backward query over all targets. On the ideal multi-permutation instantiation R1 this yields a distinguishing advantage of order $q^3/2^n$, capping the indistinguishability at $n/3$ bits. Under CTR-Perm, however, a backward lane query is not free: inverting a lane output under its prescribed counter costs an expected 2^w primitive queries, since the number of valid preimages follows $\text{Bin}(2^w, 2^{-w})$ with mean 1 (Appendix C). Balancing forward and backward queries, $\min_q \{q + (2^n/q^2) \cdot 2^w\} \approx 2^{(n+w)/3} = 2^{b/3}$, the USMA advantage becomes $O(q^3)/2^b$ in the total query count, raising the ceiling from $n/3$ to $b/3$ bits. This term is asymptotically dominant and caps the indistinguishability at $b/3$ bits.

We therefore fix $b = 1536$. The width $1536 = 6 \times 256 = 12 \times 128 = 24 \times 64$ aligns natively with the word and register granularity of mainstream computing platforms (64-bit words and 128-/256-/512-bit SIMD lanes), enabling high-speed implementations without bit-padding or lane misalignment. The resulting $b/3 = 512$ -bit indistinguishability exactly meets the design target of C-Hash-512, with $n = b - w = 1472$.

Counter Size and Message Length Constraints. Practical deployments require support for messages of arbitrary length. To accommodate messages of at least 2^{64} bits, we mandate:

$$w \geq 64 \implies w - 8 \geq 56 \text{ bits for round counters.}$$

This supports messages up to $2^{56} \times 736 > 2^{65}$ bits in length.

Permutation Size Selection. Combining the constraints yields

$$b = n + w, \quad \frac{b}{3} \gtrsim 512 \iff b \geq 1536.$$

For efficient implementation, permutation sizes are preferably multiples of 64 or 128 bits. The smallest such value satisfying $b/3 \geq 512$ with $w = 64$ is:

$$b = 1536 = 6 \times 256 = 12 \times 128 = 24 \times 64.$$

This yields concrete parameters: $n = 1472$ bits, with message block size $n/2 = 736$ bits. For $z = 1$ and the mandated maximum message length $l = 2^{56}$, the effective security is governed by the cm-pre term:

$$\log_2 q \approx \frac{b}{3} = \frac{1536}{3} = 512,$$

strictly below the quartic ceiling $552 - \log_2(9l/4)/4 \approx 538$ and the quadratic ceiling $(n - \log_2 l)/2 \approx 708$ contributed by the R1 collision analysis. We report 512-bit indistinguishability for C-Hash-512 at this maximal message length.

Engineering Alignment. The security bound demands $b \geq 1536$; we select $b = 1536$ precisely because it meets the bound with equality while being architecturally aligned: $1536 = 6 \times 256 = 12 \times 128 = 24 \times 64$. This multiples-of-64/128/256 width maps cleanly to SIMD registers (AVX-512, NEON) and hardware datapaths, eliminating awkward bit-shifting or padding overhead in both software and ASIC implementations.

Mode-Specific Optimizations for C-Hash-1024. C-Hash-1024 targets 1024-bit output with enhanced preimage resistance. Upgrading from CTR-Perm to CTR-Func fundamentally improves the security landscape: the feedforward XOR operation provably disables backward queries, improving the indistinguishability bound:

$$\mathbf{Adv}_{\text{Rocket-JH}}^{\text{indiff}}(q\mathcal{S}_i, q\mathcal{R}) \leq \frac{O(q^2l)}{2^n} + \frac{3q}{2^{n/2}}.$$

For $n = 1472$ and $l \leq 2^{56}$, the dominant term $O(q^2l)/2^n$ reaches 1 when $q \approx 2^{708}$. Thus the generic indistinguishability guarantee holds for $q \leq 2^{708}$, well above the 512-bit indistinguishability guarantee of C-Hash-512 but naturally capped by the n -bit internal state's birthday barrier.

While this generic indistinguishability bound alone would suggest $n > 2048$ is needed for full 1024-bit security, a *direct analysis* of the VIL-FIL composition reveals that C-Hash-1024 in fact achieves *full* 1024-bit preimage and second-preimage resistance. Specifically, any adversary making $q \leq 2^{1024}$ queries succeeds with probability bounded by:

$$\mathbf{Adv}_{\text{C-Hash-1024}}^{\text{pre}}(q) \leq \frac{q}{2^{1024}}, \quad \mathbf{Adv}_{\text{C-Hash-1024}}^{\text{2nd-pre}}(q) \leq \frac{O(q(l+l'))}{2^n} + \frac{O(q^2(l+l'))}{2^{3n/2}} + \frac{q}{2^{1024}}.$$

For $n = 1472$ and $l, l' \leq 2^{56}$, the VIL terms are below 2^{-103} , so the overall advantage is dominated by $q/2^{1024}$, yielding the ideal 1024-bit guarantee.

Furthermore, the un-invertibility of P'_{ctr} enables the output block size to expand to the full effective state: $h = n = 1472$ bits with $z = \lceil 1024/1472 \rceil = 1$. Thus, a single finalization permutation suffices, extracting 1024 bits via truncation of the 1472-bit state.

The 28-Round Count for C-Engine. The selection of 28 rounds for C-Engine is driven by security analysis. With differential branch number 18, each pair of consecutive rounds activates at least 18 large S-boxes. Over 14 such pairs (28 rounds), at least 252 S-boxes are guaranteed active. Given the maximum per-S-box differential probability of $2^{-8.752}$, the full 28-round differential characteristic probability is bounded by $2^{-2205.504}$, far below 2^{-1536} and thus providing a comfortable security margin.

3 Security Statements and Analyses

3.1 Concrete Security Claims

This section translates the architectural choices of Section 2 into quantifiable, provable security guarantees. Table 6 summarizes the concrete security levels achieved by the C Hash family, distinguishing between classical and quantum adversarial models.

Table 6. Security Summary for C Hash Family

Setting	Security Notion	C-Hash-512	C-Hash-1024
Classical	Indifferentiability	512	708
Classical	Preimage resistance	512	1024
Classical	Second-preimage resistance	512	1024
Classical	Collision resistance	256	512
Quantum	Preimage resistance	256	512
Quantum	Second-preimage resistance	256	512
Quantum	Collision resistance	≈ 171	≈ 341

Classical Security. The classical security bounds are derived from the rigorous analysis of the Rocket-JH framework instantiated with our 1536-bit permutation core ($n = 1472$ effective state bits).

C-Hash-512, operating in CTR-Perm mode, achieves 512-bit indifferentiability security, a ceiling imposed by the chosen multiple-preimage (USMA) barrier analyzed in Appendix C. Its 512-bit preimage and second-preimage resistance are established by direct analysis of the VIL-FIL composition (Appendix C.7), and it maintains 256-bit collision resistance optimal for a 512-bit output digest.

C-Hash-1024 upgrades to CTR-Func, which eliminates the backward-query vulnerability and the associated quartic term. Its generic indifferentiability bound is limited by the n -bit internal state’s birthday barrier to 708 bits (i.e., adversarial advantage reaches 1 at $q \approx 2^{708}$ queries). Furthermore, a direct analysis of the VIL-FIL composition (Appendix C.8) shows that the FIL finalization acts as an independent uniform random function from the n -bit VIL state to the hashlen -bit output. Consequently, C-Hash-1024 achieves full 1024-bit preimage and second-preimage resistance. Collision resistance remains governed by the birthday bound for 1024-bit outputs, i.e., 512 bits.

LDRH offers a fast path for short messages with security margins that comfortably exceed those of the standard VIL+FIL iteration. Exclusive **LaneID** assignments (0x03 for C-Hash-512, 0x83 for C-Hash-1024) enforce strict domain separation from the standard VIL and FIL lanes. Its sum-of-permutations construction achieves $2n/3 - \log_2(n/3)$ -bit indifferentiability (≈ 972 bits for $n = 1472$), significantly surpassing the 512-bit and 708-bit indifferentiability guarantees of the respective main Rocket-JH constructions. Consequently, the overall security profile remains intact.

Quantum Security. In the quantum setting, preimage and second-preimage resistance are both governed by Grover’s algorithm [12] with complexity $O(2^{\text{hashlen}/2})$. For collision resistance, the Brassard–Høyer–Tapp (BHT) algorithm [6] establishes a theoretical query upper bound of $O(2^{\text{hashlen}/3})$ in the standard quantum query model, as summarized in Table 6.

However, under realistic physical constraints—quantum memory capacity, error-correction overhead, and circuit-depth limitations—and economic cost models (time–memory product), the most practical and cost-effective attack strategy remains the classical parallel collision search of van Oorschot–Wiener [20], maintaining the birthday-bound complexity $O(2^{\text{hashlen}/2})$. These reductions are inherent to all hash functions and not construction-specific weaknesses.

Security Against Length-Extension Attacks. Length-extension attacks exploit the structural leakage of internal states in plain Merkle–Damgård: given $H(M)$, an adversary can compute $H(M \parallel \text{pad} \parallel M')$ without knowledge of M . The C Hash family is provably immune to this class of attacks by virtue of its indistinguishability from a random oracle.

Specifically, the Rocket–JH framework provides indistinguishability guarantees for arbitrarily long messages. The counter-based domain separation supports message lengths up to $l \leq 2^{w-8} = 2^{56}$ blocks, which at 736 bits per block covers inputs exceeding 2^{65} bits. C-Hash-512 achieves 512-bit indistinguishability under CTR-Perm, while C-Hash-1024 achieves 708-bit generic indistinguishability under CTR-Func; direct analysis further establishes 1024-bit preimage and second-preimage resistance for the latter. In both cases, the construction behaves as an ideal random oracle up to the corresponding query bound, so the digest $H(M)$ reveals no exploitable information about the internal chaining state $h_{l,1} \parallel h_{l,2}$.

Architecturally, the *Compress-then-Randomize* paradigm enforces strict separation between the VIL compression phase and the FIL finalization phase. The FIL finalization invokes a domain-separated permutation instance $P_{i,\text{ID}_{\text{fil}}}$ (via CTR-Perm or CTR-Func) that is cryptographically isolated from the VIL lane ID_{vil} . Consequently, even if an adversary observes the final digest, he cannot map it back to the n -bit internal state, nor can he forge a valid continuation M' by appending blocks to the absorbed chain: the counter-based domain separation ensures that the permutation queries required for the extension belong to a different, independent function family. For C-Hash-1024, the CTR-Func feedforward XOR further hardens this property by rendering the finalization function one-way. Any attempt to invert the last permutation call to recover the pre-final state $H'(M)$ requires solving $y = P_{\text{ctr}}(x) \oplus x$, which is infeasible in the public-permutation model (Appendix C). Thus, the C Hash family offers *provable* resistance to length-extension attacks for all messages within the supported length domain.

3.2 Provable Security Analysis

3.2.1 Indifferentiability Bounds We summarize the provable security guarantees for the C Hash family. The following bounds establish indifferentiability from a random oracle, assuming the underlying 1536-bit permutation P behaves ideally.

C-Hash-512 (Rocket-JH, CTR-Perm). Instantiating the Rocket-JH construction with CTR-Perm, the distinguishing advantage of any adversary making $q_{\mathcal{R}}$ queries to the construction and $q_{\mathcal{S}_i}$ queries to each of the internal permutations is bounded by

$$\mathbf{Adv}_{\text{Rocket-JH}}^{\text{indiff}}(q_{\mathcal{S}_i}, q_{\mathcal{R}}) \leq \frac{19q^2l}{2^n} + \frac{3(z-1)q^2}{2^n} + \frac{9q^4l}{2^{3n/2+2}} + \frac{O(q^3)}{2^b}, \quad (5)$$

where $q_{\mathcal{R}} \leq 2^{512}$, $q_{\mathcal{S}_i} \leq q \leq 2^{512}$, $n = b - w = 1536 - 64 = 1472$, l is the maximum message length in blocks, and $z = 1$ is the number of output blocks. The first three terms stem from the collision resistance of the VIL component and the FIL-phase analysis; the quartic term reflects the complexity of cross-round collisions under backward queries; the final term is the chosen multiple-preimage (cm-pre) barrier imposed by the Universal San Mai Attack (USMA) [15], which dominates and caps the indifferentiability at $b/3 = 512$ bits. Note that the ceiling is $b/3$ rather than $n/3$: under CTR-Perm each backward lane query costs an expected 2^w primitive queries (Appendix C), so the USMA advantage in the total query count degrades to $O(q^3)/2^b$.

C-Hash-1024 (Rocket-JH, CTR-Func). Upgrading to CTR-Func removes the backward-query vulnerability, thereby eliminating the quartic term and yielding a substantially tighter bound:

$$\mathbf{Adv}_{\text{Rocket-JH}}^{\text{indiff}}(q_{\mathcal{S}_i}, q_{\mathcal{R}}) \leq \frac{O(q^2l)}{2^n} + \frac{3q}{2^{n/2}}, \quad (6)$$

where $q_{\mathcal{R}} \leq 2^{708}$, $q_{\mathcal{S}_i} \leq q \leq 2^{708}$. For $n = 1472$ and $l \leq 2^{56}$, the dominant term reaches 1 at $q \approx 2^{708}$, which is the information-theoretic limit imposed by the n -bit internal state's birthday barrier.

LDRH (Lane-Dedicated Rapid Hash, CTR-Perm). For short messages ($|M| < n$), LDRH instantiates the sum-of-permutations $P_{\text{ctr}_1}(x) \oplus P_{\text{ctr}_2}(x)$ via two parallel CTR-Perm calls with domain-separated counters. By Corollary 1, the indifferentiability advantage is bounded by

$$\mathbf{Adv}_{\text{LDRH}}^{\text{indiff}}(q) \leq \frac{9\sqrt{n}q^{3/2}}{2^n} + \frac{48q^3}{2^{2n}} + \frac{2q+1}{2^n}, \quad (7)$$

which is asymptotically $O(\sqrt{n}q^{3/2}/2^n)$ and yields $2n/3 - \log_2(n/3)$ -bit security (i.e., security up to $q \approx 2^{972}$ queries with $n = 1472$). Dedicated **LaneID** values enforce strict domain separation from the standard multi-block lanes.

Complete proofs are given in Appendix C, and the results of the randomness statistical tests are provided in Appendix E.

3.2.2 Hash Security Guarantees By the generic reductions of Andreeva *et al.* [1], indistinguishability implies optimal preimage, second-preimage, and collision resistance:

$$\mathbf{Adv}_H^{\text{atk}}(q) \leq \mathbf{Adv}_H^{\text{indiff}}(q) + \Pr_{\mathcal{R}}[\text{atk}](q), \quad \text{atk} \in \{\text{pre}, \text{2nd-pre}, \text{coll}\},$$

where $\Pr_{\mathcal{R}}[\text{pre}](q) = q/2^{\text{hashlen}}$, $\Pr_{\mathcal{R}}[\text{2nd-pre}](q) = q/2^{\text{hashlen}}$, and $\Pr_{\mathcal{R}}[\text{coll}](q) \approx q^2/2^{\text{hashlen}+1}$.

Applying these reductions to (5) yields the concrete bounds for C-Hash-512:

$$\begin{aligned} \mathbf{Adv}_{\text{C-Hash-512}}^{\text{pre}}(q) &\leq \frac{19q^2l}{2^n} + \frac{9q^4l}{2^{3n/2+2}} + \frac{3q}{2^{n/2}} + \frac{O(q^3)}{2^b} + \frac{q}{2^{512}}, \\ \mathbf{Adv}_{\text{C-Hash-512}}^{\text{2nd-pre}}(q) &\leq \frac{19q^2l}{2^n} + \frac{9q^4l}{2^{3n/2+2}} + \frac{3q}{2^{n/2}} + \frac{O(q^3)}{2^b} + \frac{q}{2^{512}}, \\ \mathbf{Adv}_{\text{C-Hash-512}}^{\text{coll}}(q) &\leq \frac{19q^2l}{2^n} + \frac{9q^4l}{2^{3n/2+2}} + \frac{3q}{2^{n/2}} + \frac{O(q^3)}{2^b} + \frac{q^2}{2^{513}}. \end{aligned}$$

With the cm-pre term included, the generic reduction certifies these notions up to $q \leq 2^{512}$, coinciding with the 512-bit preimage and second-preimage targets of C-Hash-512. The direct analysis of Appendix C.7 independently confirms that the two notions achieve the full 512-bit level without relying on the indistinguishability ceiling. Collision resistance is governed by the 512-bit digest's birthday bound, i.e., 256 bits, and is unaffected by the cm-pre term.

For C-Hash-1024, substituting the tighter bound (6) gives

$$\begin{aligned} \mathbf{Adv}_{\text{C-Hash-1024}}^{\text{pre}}(q) &\leq \frac{O(q^2l)}{2^n} + \frac{3q}{2^{n/2}} + \frac{q}{2^{1024}}, \\ \mathbf{Adv}_{\text{C-Hash-1024}}^{\text{2nd-pre}}(q) &\leq \frac{O(q^2l)}{2^n} + \frac{3q}{2^{n/2}} + \frac{q}{2^{1024}}, \\ \mathbf{Adv}_{\text{C-Hash-1024}}^{\text{coll}}(q) &\leq \frac{O(q^2l)}{2^n} + \frac{3q}{2^{n/2}} + \frac{q^2}{2^{1025}}. \end{aligned}$$

While the above indirect bounds suggest that $n > 2048$ would be needed to guarantee full 1024-bit preimage resistance under the generic reduction, a direct analysis on the composition of the VIL and FIL components shows that C-Hash-1024 in fact achieves *full* hashlen-bit preimage and second-preimage security, see Appendix C.8. Specifically, any adversary making q queries succeeds with probability at most

$$\mathbf{Adv}_{\text{C-Hash-1024}}^{\text{pre}}(q) \leq \frac{q}{2^{1024}},$$

and

$$\mathbf{Adv}_{\text{C-Hash-1024}}^{\text{2nd-pre}}(q) \leq \frac{O(q(l+l'))}{2^n} + \frac{O(q^2(l+l'))}{2^{3n/2}} + \frac{q}{2^{1024}}.$$

For $n = 1472$ and $l, l' \leq 2^{56}$, the VIL terms are below 2^{-103} , so the overall advantage is dominated by $q/2^{1024}$, yielding the ideal 1024-bit guarantee.

3.3 Security Analysis of C-Engine Permutaion

We have evaluated the security margins of C-Engine permutaion against differential distinguisher, linear distinguisher, zero-sum distinguisher, algebraic attack, and structural attacks currently known. Under the ideal-permutation model, the construction is indistinguishable from a random 1536-bit permutation within the query regimes relevant to practical security.

3.3.1 Security Analysis of Components

The 4-bit S-box. The 4-bit S-box has zero fixed points and attains optimal differential uniformity (maximum DDT entry 4), optimal maximum LAT entry 4, maximum SAC deviation 4, and algebraic degree 3. Its boomerang connectivity table satisfies $\text{BCT}_{\max} = 6$ with no entries ≥ 8 (i.e., no boomerang transition with probability $\geq 1/2$).

The 24-bit S-box. The 24-bit S-box adopts an SPS (Substitution-Permutation-Substitution) structure: two layers of 4-bit S-box separated by a 6×6 binary matrix over $\text{GF}(2^4)$ with branch number 4 and an internal constant derived from $\sqrt{2}$. Exact evaluation yields a maximum differential probability of $p_{\max} = 2^{-8.75}$ and a maximum linear correlation of $c_{\max} = 2^{-6.19}$.

The $\mathcal{M}$ -Layer. The 64×64 binary matrix M_{64} is constructed via a 4-round extended Lai-Massey structure achieving differential branch number and linear branch number both equal to 18, which guarantee strong diffusion effect.

Round Constants C-Engine employs twenty-eight distinct round constants derived from π and an S-box internal constant derived from $\sqrt{2}$, which is independent of the round constants. These constants mitigate slide attacks, rotational attacks, invariant subspace attack and other symmetry-exploiting techniques.

3.3.2 Differential Cryptanalysis The resistance against differential cryptanalysis is quantified through the maximum differential characteristic probability over the full 28 rounds.

Minimum Number of Active S-Boxes. The linear binary matrix M_{64} achieves differential branch number $B_d = 18$. By the definition of branch number, any non-zero differential pattern passing through the $\mathcal{M}$ -layer activates at least 18 S-boxes for any two rounds. Therefore, over 28 rounds (comprising 14 two-round structures), the minimum number of active S-boxes is:

$$N_{\text{active}} = \frac{28}{2} \times 18 = 14 \times 18 = 252. \quad (8)$$

Maximum Differential Characteristic Probability. Considering the maximum differential probability of the 24-bit S-box is $p_{\max} = 2^{-8.75}$ (as determined by exhaustive search). The maximum differential characteristic probability over 28 rounds is therefore bounded by:

$$P_{\max}^{(28)} \leq \left(2^{-8.75}\right)^{252} = 2^{-8.75 \times 252} = 2^{-2205}. \quad (9)$$

Since the differential probability $2^{-2205} \ll 2^{-1536}$, there is no useful differential characteristic can be utilized to distinguish the full C-Engine permutation. This bound provides overwhelming resistance against differential attacks

3.3.3 Linear Cryptanalysis The resistance against linear cryptanalysis follows the same principle of differential analysis. The linear branch number of the diffusion matrix equals 18 (coinciding with the differential branch number). Therefore, there are at least 18 active S-boxes for any two rounds. For the full 28 rounds (comprising 14 two-round structures), the minimum number of active S-boxes is also 252. Considering that the maximum linear correlation of the 24-bit S-box is $c_{\max} = 2^{-6.19}$ (as determined by exhaustive search), the maximum linear trail correlation is bounded by $c_{\max}^{(28)} \leq (2^{-6.19})^{252} = 2^{-1560}$, providing overwhelming resistance against linear cryptanalysis.

3.3.4 Zero-Sum Distinguisher Zero-sum distinguishers exploit the algebraic degree of the permutation. For C-Engine round function, the forward and backward algebraic degrees are both 9. This can be determined by the SPS structure of the 24-bit S-box, since the algebraic degree of the small 4-bit S-box is 3. :

$$9 \times 9 \times 9 = 9^3 = 729. \quad (10)$$

Therefore, both the forward and backward maximum number of rounds vulnerable to zero-sum distinguishers are all 3-round. This implies that zero-sum distinguishers can exist for at most 6 rounds, leaving a security margin of $28 - 6 = 22$ rounds. For the full 28-round C-Engine permutation, we believe that there is no effective zero-sum distinguisher.

3.3.5 Algebraic Attacks The 24-bit S-box achieves an algebraic degree of 9, which is the maximum attainable under the SPS structure. After 28 rounds of interleaved non-linear and linear transformations, the resulting system of polynomial equations describing C-Engine exhibits prohibitively high degree and dense cross-variable coupling. This renders Gröbner basis attacks and interpolation attacks computationally infeasible. The large state size (1536 bits) further amplifies the memory and time complexity of any algebraic approach.

4 Performance Evaluation

4.1 Performance Analyses

The performance of the C Hash family is evaluated from both algorithmic and implementation perspectives, leveraging the structural parameters of the Rocket-JH mode and the C-Engine permutation core.

Long-Message Throughput. For messages of length $|M| \geq n = 1472$ bits, the Rocket-JH iteration operates in the VIL compression phase with an effective block size of $n/2 = 736$ bits absorbed per permutation call. Under the 1536-bit C-Engine core ($b = 1536$), this yields a raw *absorption ratio* of

$$\frac{736}{1536} \approx 47.9\%,$$

i.e., nearly half of the 1536-bit permutation width carries message entropy per call. This ratio significantly exceeds the typical rate of homogeneous sponge constructions under comparable security margins, translating directly into higher throughput for large data streams.

Short-Message Latency. For $|M| < n = 1472$ bits, both instances bypass the iterative Rocket-JH pipeline and invoke the Lane-Dedicated Rapid Hash (LDRH). LDRH pads the message to n bits ($M \parallel 1 \parallel 0^{n-1-|M|}$) and evaluates two parallel CTR-Perm calls with distinct counters ($\text{ctr}_1, \text{ctr}_2$), combining their n -bit outputs by XOR. Although this entails two C-Engine evaluations, it eliminates the initialization overhead, the multi-round VIL chaining, and the separate FIL finalization phase of standard wide-pipe constructions. The resulting latency is therefore bounded by a small constant (2 permutation calls) regardless of message length, providing uniformly low response times for packet authentication, handshake transcripts, and commitment schemes.

C-Engine Core Cost. Each invocation of the underlying C-Engine permutation processes a 1536-bit state through 28 rounds. Within each round, the $\mathcal{S}$ -Layer executes 64 parallel 24-bit S-boxes, each applying a 6×6 diffusion matrix over $\text{GF}(2^4)$; the $\mathcal{M}$ -Layer applies the 64×64 matrix via a 4-round extended Lai-Massey structure. The state is organized as a 6×64 nibble matrix, comprising 384 nibbles (1536 bits in packed representation). On 64-bit platforms, this compact state maps naturally to 24 machine words. For vectorized S-box evaluation, nibbles are expanded to byte granularity in registers, enabling direct parallel lookups via $\text{PSHUFb}/\text{VPSHUFb}$; with AVX2 (256-bit SIMD), the byte-unpacked state occupies only 12 YMM registers. Consequently, a single C-Engine round reduces to a sequence of parallel table lookups and bitwise XOR operations on contiguous register vectors, delivering high throughput with low register pressure and no memory traffic.

Comparative Assessment. When normalized by the 1536-bit primitive width, the Rocket-JH mode achieves a Hash Effectiveness (H.E.) of approximately 0.1597 ($512 \times 736/1536^2$; see H.E. definition in [22]), breaking the $1/8$ ceiling inherent to homogeneous sponge and Merkle-Damgård constructions. This structural efficiency, combined with the constant-time LDRH fast path, establishes C Hash as a high-throughput, low-latency hash family suitable for both bandwidth-intensive long-message workloads and latency-critical short-message applications.

4.2 Performance Test

4.2.1 Detailed Configuration of Implementation Platforms

Implementation Method. The C-Hash algorithm family is implemented in the C programming language (ISO C99). Two build profiles are provided for each algorithm instance:

- **Reference (REF):** Portable, standard-compliant C code compiled with `-std=c99 -Wpedantic -Wall -Wextra -O2`.
- **Optimized (OPT):** High-performance C code utilizing x86-64 AVX2 vector intrinsics, compiled with `-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer`.

All builds use GCC 12.2.0 with position-independent code (`-fPIC`) and are self-contained (no external crypto libraries).

Software Implementation Platform. The software implementation platform configuration is summarized in Table 7.

Table 7. Software Implementation Platform Configuration

Parameter	Configuration
Processor model	Intel Xeon Platinum (Family 6, Model 85)
Architecture	x86_64 (64-bit PC processor)
Clock frequency	2.5 GHz
Instruction set extensions	x86-64 baseline, AVX2, AVX-512F/DQ/CD/BW/VL
Core utilization	Single core (pinned to core 0, SMT disabled for measurement)
L3 cache	33 MiB
Memory (RAM)	4.0 GiB
Operating system	Debian GNU/Linux 12 (bookworm)
Compiler	GCC 12.2.0
Build system	GNU Make 4.3
Crypto libraries	None (self-contained implementation)

4.2.2 Performance Test Results

Software Implementation Efficiency. Performance was measured on the S1–S8 data levels specified in the Self-Assessment Guidelines. Each reported value is the trimmed mean of 128 iterations using RDTSCP timing with CPUID serialization. Table 8 reports throughput in MB/s; Table 9 reports computational efficiency in cycles per byte.

Table 8. Throughput (MB/s) by Instance and Message Length

Instance	S1	S2	S3	S4	S5	S6	S7	S8
	32 B	128 B	512 B	1 024 B	4 096 B	8 192 B	16 384 B	65 536 B
C-Hash-512-REF	0.19	0.78	0.77	0.88	0.97	0.71	0.93	0.84
C-Hash-512-OPT	11.22	44.94	45.00	51.80	63.32	62.73	64.25	64.41
C-Hash-1024-REF	0.20	0.79	0.78	0.89	1.06	1.08	0.95	0.84
C-Hash-1024-OPT	11.20	44.85	44.60	51.32	59.70	61.30	62.13	62.52

Table 9. Computational Efficiency (Cycles/Byte) by Instance and Message Length

Instance	S1	S2	S3	S4	S5	S6	S7	S8
C-Hash-512-REF	12,322.3	3,041.1	3,077.8	2,704.9	2,463.6	3,362.4	2,560.6	2,834.9
C-Hash-512-OPT	212.5	53.1	53.0	46.0	37.7	38.0	37.1	37.0
C-Hash-1024-REF	12,133.0	3,030.7	3,045.0	2,672.2	2,249.8	2,214.6	2,521.1	2,843.3
C-Hash-1024-OPT	212.8	53.2	53.5	46.5	39.9	38.9	38.4	38.1

Software Resource Consumption (Optional). Table 10 summarizes the compiled binary size (static memory) and peak runtime memory footprint measured during S1–S8 testing.

Table 10. Software Resource Consumption

Instance	Static Memory (B)	Peak Memory S1–S7 (kB)	Peak Memory S8 (kB)
C-Hash-512-REF	21,875	2,476	2,612
C-Hash-512-OPT	33,902	2,484	2,620
C-Hash-1024-REF	22,003	2,476	2,612
C-Hash-1024-OPT	34,166	2,484	2,620

Note: Static memory is the sum of text, data, and BSS segments measured by the `size` utility. Peak memory is the `VmPeak` value from `/proc/self/status`.

5 Feature Statements

5.1 Innovativeness

5.1.1 Unified Decoupling Framework C Hash presents the first hash-function family that unifies three decoupling principles—structural (VIL vs. FIL), temporal (pairwise distinct round functions), and implementation-level (domain-separated multiplexing)—within a single, coherent construction. Unlike prior designs that recirculate a single primitive through both absorption and finalization, C Hash architecturally decouples these phases, enabling the VIL layer to maximize throughput while the FIL layer independently guarantees random-oracle behavior. The Rocket-JH iteration simultaneously achieves high indistinguishability and high efficiency, quantified by the Hash Effectiveness (H.E.) metric that fundamentally overcomes the $\text{H.E.} \leq 1/8$ barrier inherent to monolithic sponge or Merkle-Damgård paradigms.

5.1.2 Counter-Based Domain Multiplexing A distinctive engineering contribution is the CTR-Perm/CTR-Func mechanism, which resolves the apparent contradiction between the theoretical need for pairwise distinct permutations and the practical cost of instantiating them. By embedding a w -bit counter into a single public permutation $P: \{0,1\}^b \rightarrow \{0,1\}^b$, C Hash derives 2^w cryptographically isolated function instances without key schedules, round-constant tables, or multiple cores. This multiplexed-unity architecture enables both algorithm variants (C-Hash-512 and C-Hash-1024) to share identical silicon while differing only in a single XOR gate and counter suffix, achieving mode-select security upgrades at negligible implementation cost. Beyond the core VIL and FIL lanes, the 2^8 -wide LaneID space further reserves dedicated channels for LDRH (short-message fast path), LDXOF (extendable output), and future application-specific modes, all cryptographically isolated from the main hash pipeline.

5.1.3 Mandatory Low-Latency Fast Path C Hash mandates an automatic short-message fast path (LDRH) that bypasses the iterative VIL+FIL pipeline entirely for inputs shorter than the effective block size. This is not an optional optimization but a normative component of the specification, enforced by dedicated, cryptographically isolated lane identifiers. The resulting dual-path architecture guarantees uniformly low latency across the entire input-length spectrum, eliminating the initialization penalty that penalizes standard wide-pipe constructions on prevalent short-message workloads such as packet authentication and commitment schemes.

5.2 Simplicity

5.2.1 Counter-Derived Round Functions From an implementation perspective, the domain-separation mechanism collapses all operational variants—initialization, VIL compression, FIL finalization, and even the LDRH/LDXOF fast paths—into a single, branch-free invocation of the C-Engine permutation. The round index, lane identifier, and instance counter are packed into

a w -bit field appended to the payload; no key schedule, round-constant ROM, or runtime table lookup is ever required.

Consequently, a compliant implementation consists of one unified permutation routine whose sole variable input is the monotonically increasing counter. Initialization, message absorption, and hash extraction all execute the same code path, differing only in the counter assignment. This eliminates branch-heavy special-case logic, minimizes code footprint (the core loop fits easily in instruction cache), and removes data-dependent memory accesses, yielding a constant-time implementation with negligible side-channel leakage.

5.2.2 Minimalist IV Design The initial value is constructed as the binary encoding of the target hash length, $IV_{\text{hashlen}} = [\text{hashlen}]_{n/2}$, requiring no algorithm-specific constant tables. Moreover, because IV_{hashlen} is known at compile time, the first-round permutation call on the IV can be precomputed and hardcoded, reducing the online initialization overhead to a trivial XOR operation.

5.2.3 Unified Construction Rocket-JH employs a unified JH iteration structure for both IV initialization and message compression—the same permutation-based transformation handles the initial state setup and all subsequent message block injections, eliminating special-case processing logic and ensuring architectural symmetry. Building upon this internal uniformity, the C Hash family instantiates two algorithms—C-Hash-512 and C-Hash-1024—from a single framework. The sole distinction lies in the domain-separation mode (CTR-Perm versus CTR-Func), enabling both variants to share an identical implementation codebase with only minimal mode-specific branching.

5.3 Flexibility

5.3.1 Parameterizable Permutation Support The relationship $b = n + w$ allows trade-offs between security margin and auxiliary data capacity. The counter space ($w \geq 64$ bits) provides ample room for personalization strings or application-specific domains.

5.3.2 Extensible Cryptographic Primitives The Rocket-JH framework naturally extends beyond basic hashing, with each mode inheriting the provable indistinguishability security of the decoupling design:

- **MAC**: Incorporating a secret key into the IV yields a secure MAC construction.
- **KDF**: Domain-separated structure supports hierarchical key derivation.
- **XOF**: The finalization phase supports arbitrary output lengths by increasing the number of extraction rounds z ; moreover, the dedicated LDXOF mode (Appendix B) provides optimized extendable-output functionality for short seeds.

5.3.3 Dedicated Mechanism for Short Messages A specialized **Lane-Dedicated Rapid Hash (LDRH)** optimizes the dominant case of short inputs ($< n$ bits, i.e., < 1472 bits or 184 bytes for C-Hash instances). By bypassing the iterative absorption and finalization phases, LDRH eliminates the initialization

and multi-round overhead that penalizes standard VIL constructions on short inputs. Reserved `LaneID` values (`0x03` for C-Hash-512, `0x83` for C-Hash-1024) provide strict domain separation from standard multi-block lanes, ensuring that C Hash delivers both optimal latency for critical short-message paths and high throughput for bandwidth-intensive long messages.

5.4 Performance Advantages

5.4.1 Superior Hash Effectiveness The efficacy of decoupling is formally quantified by *Hash Effectiveness* (H.E.) [22], a scale-invariant metric capturing the Pareto efficiency of security-efficiency trade-offs:

$$\text{H.E.} = \underbrace{\frac{\text{Security Level}}{b}}_{\text{Security Ratio}} \times \underbrace{\frac{\text{Processing Rate}}{b}}_{\text{Efficiency Ratio}}.$$

Constructions like sponge and MD are fundamentally capped at $\text{H.E.} \leq 1/8$, because homogeneous designs typically achieve at most $c/2$ bits of indistinguishability security while absorbing r bits per call, giving $\text{H.E.} \leq (c/2 \cdot r)/b^2$. For the balanced case $r = c = b/2$, this yields the ceiling $\text{H.E.} \leq 1/8$. By contrast, C Hash’s decoupled architecture breaks through this ceiling. For Rocket-JH, instantiated with a 1536-bit permutation core:

- **Security Level:** 512-bit indistinguishability against distinguishing attacks;
- **Processing Rate:** 736 bits absorbed per permutation call;
- **Hash Effectiveness:** $\frac{512 \times 736}{1536^2} \approx 0.1597$.

This quantitative confirmation demonstrates that architectural decoupling is not merely a theoretical abstraction but a practical design principle that fundamentally expands the achievable Pareto frontier of hash function design.

5.4.2 Nibble-Sliced Bit-Parallel Architecture The defining performance advantage of C-Engine lies in its nibble-sliced design, which processes all 384 state nibbles simultaneously through bit-slicing. Each of the 4 bit planes is stored and manipulated as a contiguous machine word (64-bit or 128-bit), enabling a single 4-bit S-box evaluation to operate on 384 bits in parallel. This architecture maps naturally to SIMD instructions on modern processors, delivering exceptional throughput without dedicated vector units. On 64-bit platforms, the entire S-box layer reduces to a handful of bitwise operations on 24 machine words; on AVX2/NEON, the 128-bit word width halves this to 12 words ($1536/128 = 12$).

References

1. Andreeva, E., Mennink, B., Preneel, B.: Security reductions of the second round SHA-3 candidates. In: Burmester, M., Tsudik, G., Magliveras, S.S., Ilic, I. (eds.) *Information Security - 13th International Conference, ISC 2010, Boca Raton, FL, USA, October 25-28, 2010, Revised Selected Papers. Lecture Notes in Computer Science*, vol. 6531, pp. 39–53. Springer (2010), https://doi.org/10.1007/978-3-642-18178-8_5
2. Bellare, M., Rogaway, P.: Random oracles are practical: A paradigm for designing efficient protocols. In: Denning, D.E., Pyle, R., Ganesan, R., Sandhu, R.S., Ashby, V. (eds.) *CCS '93, Proceedings of the 1st ACM Conference on Computer and Communications Security*, Fairfax, Virginia, USA, November 3-5, 1993. pp. 62–73. ACM (1993), <https://doi.org/10.1145/168588.168596>
3. Bertoni, G., Daemen, J., Peeters, M., Assche, G.V.: On the indistinguishability of the sponge construction. In: Smart, N.P. (ed.) *Advances in Cryptology - EUROCRYPT 2008, 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Istanbul, Turkey, April 13-17, 2008. Proceedings. *Lecture Notes in Computer Science*, vol. 4965, pp. 181–197. Springer (2008), https://doi.org/10.1007/978-3-540-78967-3_11
4. Bhattacharya, S., Nandi, M.: Full indistinguishable security of the XOR of two or more random permutations using the χ^2 method. In: *Advances in Cryptology - EUROCRYPT 2018, Part I. Lecture Notes in Computer Science*, vol. 10820, pp. 387–412. Springer (2018). https://doi.org/10.1007/978-3-319-78381-9_15
5. Bhaumik, R., Datta, N., Dutta, A., Jha, A., Mandal, S., Mennink, B., Nandi, H., Shen, Y.: How to build a short-input random oracle from public random permutations. In: Daemen, J., Thomé, E. (eds.) *Advances in Cryptology - EUROCRYPT 2026 - 45th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Rome, Italy, May 10-14, 2026, Proceedings, Part VI. pp. 394–423. *Lecture Notes in Computer Science*, Springer (2026), https://doi.org/10.1007/978-3-032-25333-0_14
6. Brassard, G., Høyer, P., Tapp, A.: Quantum cryptanalysis of hash and claw-free functions. In: *LATIN '98: Theoretical Informatics, Third Latin American Symposium. Lecture Notes in Computer Science*, vol. 1380, pp. 163–169. Springer (1998), <https://doi.org/10.1007/BFb0054319>
7. Chen, Y.L.: A modular approach to the security analysis of two-permutation constructions. In: *Advances in Cryptology - ASIACRYPT 2022, Part I. Lecture Notes in Computer Science*, vol. 13791, pp. 379–409. Springer (2022). https://doi.org/10.1007/978-3-031-22963-9_13
8. Coron, J., Dodis, Y., Malinaud, C., Puniya, P.: Merkle-damgård revisited: How to construct a hash function. In: Shoup, V. (ed.) *Advances in Cryptology - CRYPTO 2005: 25th Annual International Cryptology Conference*, Santa Barbara, California, USA, August 14-18, 2005, Proceedings. *Lecture Notes in Computer Science*, vol. 3621, pp. 430–448. Springer (2005), https://doi.org/10.1007/11535218_26
9. Damgård, I.: A design principle for hash functions. In: Brassard, G. (ed.) *Advances in Cryptology - CRYPTO '89, 9th Annual International Cryptology Conference*, Santa Barbara, California, USA, August 20-24, 1989, Proceedings. *Lecture Notes in Computer Science*, vol. 435, pp. 416–427. Springer (1989), https://doi.org/10.1007/0-387-34805-0_39
10. Dinur, I.: Combining outputs of a random permutation: New constructions and tight security bounds by fourier analysis. In: *Advances in Cryptology - EUROCRYPT 2025, Part I. Lecture Notes in Computer Science*, vol. 15601, pp. 244–273. Springer (2025). https://doi.org/10.1007/978-3-031-91098-9_9
11. Dodis, Y., Ristenpart, T., Shrimpton, T.: Salvaging merkle-damgård for practical applications. In: Joux, A. (ed.) *Advances in Cryptology - EUROCRYPT 2009, 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Cologne, Germany, April 26-30, 2009. Proceedings. *Lecture Notes in Computer Science*, vol. 5479, pp. 371–388. Springer (2009), https://doi.org/10.1007/978-3-642-01001-9_22
12. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing*. pp. 212–219. ACM (1996), <https://doi.org/10.1145/237814.237866>
13. Günsing, A., Bhaumik, R., Jha, A., Mennink, B., Shen, Y.: Revisiting the indistinguishability of the sum of permutations. In: *Advances in Cryptology - CRYPTO 2023, Part III. Lecture Notes in Computer Science*, vol. 14083, pp. 628–660. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_21
14. Iwata, T.: New blockcipher modes of operation with beyond the birthday bound security. In: Robshaw, M.J.B. (ed.) *Fast Software Encryption, 13th International Workshop, FSE 2006, Graz, Austria, March 15-17, 2006, Revised Selected Papers*. pp. 310–327. *Lecture Notes in Computer Science*, Springer (2006), https://doi.org/10.1007/11799313_20
15. Li, H., Li, C., Hu, H., Liu, F.: Indistinguishability of the sponge hash family: New attacks and the optimal construction. *IACR Transactions on Symmetric Cryptology* **2026**(1), 31–75 (2026). <https://doi.org/10.46586/tosc.v2026.i1.31-75>

-
16. Lucks, S.: The sum of prps is a secure prf. In: Advances in Cryptology - EUROCRYPT 2000, International Conference on the Theory and Application of Cryptographic Techniques, Bruges, Belgium, May 14-18, 2000, Proceeding. Lecture Notes in Computer Science, vol. 1807, pp. 470–484. Springer (2000). https://doi.org/10.1007/3-540-45539-6_34
 17. Maurer, U.M., Renner, R., Holenstein, C.: Indifferentiability, impossibility results on reductions, and applications to the random oracle methodology. In: Naor, M. (ed.) Theory of Cryptography, First Theory of Cryptography Conference, TCC 2004, Cambridge, MA, USA, February 19-21, 2004, Proceedings. Lecture Notes in Computer Science, vol. 2951, pp. 21–39. Springer (2004), https://doi.org/10.1007/978-3-540-24638-1_2
 18. Merkle, R.C.: One way hash functions and DES. In: Brassard, G. (ed.) Advances in Cryptology - CRYPTO '89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings. Lecture Notes in Computer Science, vol. 435, pp. 428–446. Springer (1989), https://doi.org/10.1007/0-387-34805-0_40
 19. Moody, D., Paul, S., Smith-Tone, D.: Improved indifferentiability security bound for the JH mode. Des. Codes Cryptogr. **79**(2), 237–259 (2016), <https://doi.org/10.1007/s10623-015-0047-9>
 20. van Oorschot, P.C., Wiener, M.J.: Parallel collision search with cryptanalytic applications. J. Cryptology **12**(1), 1–28
 21. Wu, H.: The Hash Function JH. <https://csrc.nist.gov/projects/hash-functions/sha-3-project>
 22. Zhang, L., Sui, H., Zhang, L., Wu, W.: How to construct even faster and indifferentiable hash functions from random permutations. Cryptology ePrint Archive, Paper 2026/713 (2026), <https://eprint.iacr.org/2026/713>

A Implementation Notes and Clarifications

This appendix provides concrete implementation guidance to resolve ambiguities that may arise when translating the C Hash specification into executable code. All recommendations herein are *normative* for compliant implementations.

Byte Order (Endianness). The specification employs multi-byte encodings for the initial value IV_{hashlen} , domain-separation counters ctr , and round constants $RC^{(r)}$. To ensure bitwise interoperability across platforms, all such quantities are encoded in **big-endian (network) order**, with the following conventions:

- The most significant byte occupies the lowest memory address (leftmost position in a byte sequence).
- Within each byte, bit indices are numbered from 7 (MSB) down to 0 (LSB).

Example 1 (IV encoding). The 736-bit encoding of $IV_{512} = 512 = 0x0200$ consists of 92 bytes. In big-endian layout, the byte $0x02$ resides at offset 90 and $0x00$ at offset 91, with all preceding bytes set to zero.

Example 2 (Counter encoding). A 64-bit counter $\text{ctr} = 0x0100000000000003$ (LaneID $0x01$, round index 3) is stored as an 8-byte sequence where $0x01$ occupies the lowest-address byte and $0x03$ occupies the highest-address byte.

Implementations running on little-endian hardware must perform explicit byte-order conversion (e.g., `htonl`-style routines or byte-swap operations) when serializing or deserializing these values.

Unified Counter Layout. The exact counter payload allocation for all C Hash modes is given in Table 1; we repeat below only the implementation-specific layout remarks.

All C Hash modes use the following invariant 64-bit format:

$$\text{ctr} = [\text{LaneID}]_8 \parallel [\text{Payload}]_{56},$$

where $[\text{LaneID}]_8$ occupies bits 63–56 (most-significant byte) and $[\text{Payload}]_{56}$ occupies bits 55–0. The **LaneID** field provides algorithm-level domain separation; the remaining 56 bits are mode-specific. Placing **LaneID** in the most-significant byte guarantees that carry propagation from incrementing the lower 56 bits can never alter the algorithm identity, preserving strict cryptographic isolation across all lanes.

Bit-Level Message Padding. The formal padding specification is given in Section 1.2.2. The following example clarifies the byte-level bit placement for implementations.

Example. Let $r = |M| \bmod 8$ be the number of occupied bits in the last partial byte (numbered 7 down to 0). The padding step writes a 1 into bit position $7 - r$ and zeros into positions $6 - r$ down to 0 of the same byte; additional all-zero bytes are appended until the total length is a multiple of 736 bits.

Suppose $|M| = 10$ and the message consists of ten 1 bits. Byte 0 is `1111 1111` and byte 1 contains the remaining two message bits in positions 7 and 6 (i.e., `1100 0000`). The padding 1 is written into bit 5 of byte 1, yielding `1110 0000`. The remaining 725 bits of the 736-bit block are zero.

Empty-String Handling in FIL Finalization. Algorithm 1 defines the FIL phase as extracting $h_{l+i,1}$ (length h) and $h_{l+i,2}$ (length $n - h$) from the n -bit permutation output. For C-Hash-1024, Table 5 sets $h = n = 1472$, which implies $n - h = 0$.

In this case, $h_{l+i,2}$ is the **empty string** ε , and the concatenation $h_{l+i,1} \parallel h_{l+i,2}$ reduces to $h_{l+i,1}$ alone. Implementations must handle $h = n$ gracefully without attempting to read or write zero-length buffers. A compliant implementation may therefore treat the FIL phase as a single n -bit truncation when $h = n$.

Mandatory LDRH Short-Message Routing. Short-message routing to the LDRH lane is mandatory; see Section 1.2.5 and Algorithm 2. For any input message M with $|M| < n$ bits ($n = 1472$), the implementation **MUST** invoke the dedicated LDRH lane (LaneID 0x03 for C-Hash-512, 0x83 for C-Hash-1024) instead of the standard two-phase Rocket-JH iteration.

Leftmost-Bit Extraction (LMB). The function $\text{LMB}(a, x)$ extracts the leftmost (most significant) a bits of x . For software implementations that operate on byte-aligned buffers, the exact behavior when a is not a multiple of 8 is defined as follows:

- Let $b = \lceil a/8 \rceil$ be the number of bytes required.
- The implementation copies the first b bytes of x .
- Let $r = a \bmod 8$. If $r \neq 0$, the last byte (byte $b - 1$) contributes only its r most-significant bits. Its remaining $8 - r$ low-order bits are masked to zero by bitwise-AND with a mask whose highest r bits are 1 and lowest $8 - r$ bits are 0. For example, when $r = 4$ the mask is 0xF0.

Example. $\text{LMB}(500, x)$ where x is 1472 bits: $b = 63$ bytes and $r = 4$. The first 62 bytes are copied verbatim. The 63rd byte (index 62) retains only its 4 most-significant bits; its 4 least-significant bits are cleared with mask 0xF0. The resulting 63-byte string contains exactly 500 meaningful bits.

B The Lane-Dedicated eXtendable Output Function for Short Messages

To address the demand for low-latency extendable-output hashing on short messages, the C Hash family introduces the **Lane-Dedicated eXtendable Output Function** (LDXOF). This mode is purpose-built for inputs satisfying $|M| < n$, bypassing the full two-phase Rocket-JH iteration (VIL absorption plus FIL finalization) while retaining provable indistinguishability guarantees. LDXOF occupies a dedicated lane identifier that is cryptographically isolated from the standard multi-block hash path and the LDRH short-message hash path. Concretely, we reserve 0x04 for C-Hash-512 and 0x84 for C-Hash-1024, ensuring that no cross-lane query cycles can occur within the CTR-Perm/CTR-Func framework.

Cryptographic Core. LDXOF instantiates the XORP (XOR of Permutations) paradigm, originally introduced by Lucks [16] as the sum of PRPs and later adopted as the core of CENC [14], and recently revisited in the public-permutation setting by Bhaumik et al. [5]. Through the domain-separated CTR-Perm interface, a single

public permutation $P : \{0,1\}^b \rightarrow \{0,1\}^b$ is multiplexed into $l + 1$ effectively independent function instances $\{P_{\text{ctr}_i}\}_{i=0}^l$. The construction draws a baseline output y_0 and expands it into l pairwise XOR-differences

$$r_i = y_0 \oplus y_i \quad \text{for } i = 1, \dots, l,$$

where each y_i is the n -bit truncation of $P(x \parallel \text{ctr}_i)$. This sum-of-permutations structure strips the algebraic artifacts (fixed points, invertibility, and cyclic behavior) inherent to a single permutation, yielding an ideal fixed-input-length random function. Specifically, LDXOF is defined in Alg. 4 below.

Algorithm 4: THE LANE-DEDICATED XOF (LDXOF)

Input: Message $M \in \{0,1\}^*$ with $|M| < n$, target output length $\text{OutputByteLen} \leq ln/8$, dedicated lane identifier $\text{LaneID} \in \{0,1\}^8$, Permutation $P: \{0,1\}^b \rightarrow \{0,1\}^b$ where $b = n + w$

Output: $\text{output} \in \{0,1\}^{\text{OutputByteLen}}$

```

1  $x \leftarrow M \parallel 1 \parallel 0^{n-1-|M|}$ ; //  $n$ -bit padded message
2 for  $i = 0$  to  $l$  do
3    $\text{ctr}_i \leftarrow [\text{LaneID} \parallel \text{OutputByteLen} \parallel 0x00 \parallel i]_w$ ; //  $w$ -bit counter for the  $i$ -th instance;  $0x00$  is
   the 8-bit Reserved field
4    $y_i \leftarrow \text{LMB}(n, P(x \parallel \text{ctr}_i))$ ; //  $i$ -th CTR-Perm call
5 for  $i = 1$  to  $l$  do
6    $r_i \leftarrow y_0 \oplus y_i$ ; // XOR of baseline  $y_0$  and  $i$ -th output  $y_i$ 
7  $\text{output} \leftarrow \text{LMB}(\text{OutputByteLen} \times 8, r_1 \parallel r_2 \parallel \dots \parallel r_l)$ ;
8 return  $\text{output}$ ;
```

Indifferentiability. The indifferentiability of XORP has been established by a line of research culminating in tight bounds. Adapting Theorem 10 of Bhaumik et al. [5] and replacing their instance-count parameter w with the LDXOF block parameter l , we obtain: for any distinguisher $\mathcal{D}$ making at most q queries provided $9n \leq 2q \leq 2^n - 1$, there exists an efficient simulator $\mathcal{S}$ such that

$$\text{Adv}_{\text{LDXOF}, \mathcal{S}}^{\text{indiff}}(\mathcal{D}) \leq \frac{9l^2 q \sqrt{nq}}{2^n} + \frac{48l^4 q^3}{2^{2n}} + \frac{2l^2 q}{2^n} + \frac{l^2}{2^n}. \quad (11)$$

This bound places the construction in the beyond-birthday-bound regime, delivering approximately $2n/3$ bits of indifferentiability security. With C Hash’s concrete parameter $n = 1472$, the security margin comfortably exceeds the 512-bit (resp. 708-bit) indifferentiability targets of C-Hash-512 (resp. C-Hash-1024).

Counter Layout. In the C Hash instantiation, the 64-bit counter is partitioned into four byte-aligned fields, with the instance index placed in the least-significant bits to facilitate efficient sequential incrementing:

- **LaneID** (8 bits, bits 63–56): Algorithm-level domain separation. Values $0x04$ and $0x84$ are reserved exclusively for LDXOF in C-Hash-512 and C-Hash-1024, respectively.

- **OutputByteLen** (24 bits, bits 55–32): Encodes the target output length in bytes, supporting arbitrary outputs up to $2^{24} - 1$ bytes (16 MiB).
- **Reserved** (8 bits, bits 31–24): Fixed to zero; serves as a buffer to prevent carry propagation from the instance index into the OutputByteLen field during counter increment.
- **Instance index i** (24 bits, bits 23–0): Serves as the round counter for CTR-Perm. Placed in the least-significant position, it supports sequential incrementing via a single integer addition without masking or shift operations.
The structured counter is encoded as

$$\text{ctr}_i = [\text{LanelD}]_8 \parallel [\text{OutputByteLen}]_{24} \parallel [\text{Reserved}]_8 \parallel [i]_{24}.$$

With this layout, the low 32 bits (Reserved + i) form a naturally aligned word in which i can be incremented independently of the upper fields, yielding optimal efficiency in both software and hardware implementations.

C Security Proofs for Rocket-JH

C.1 Security Definitions

A *random permutation* P over $\{0, 1\}^n$ is a permutation selected uniformly at random from the set of all $(2^n)!$ permutations on n -bit strings; its inverse is denoted P^{-1} . A *random function* RF from $\{0, 1\}^*$ to $\{0, 1\}^n$ is a function where each distinct input x is assigned an output sampled independently and uniformly from $\{0, 1\}^n$.

A *hash function* is a deterministic algorithm $H : \{0, 1\}^* \rightarrow \{0, 1\}^{\text{hashlen}}$ that maps arbitrary-length messages to fixed-length digests of `hashlen` bits. The security notions for hash functions originate from the seminal works of Merkle and Damgård [9,18]. This paper focuses on the following fundamental properties: collision resistance, preimage resistance, multiple-preimage resistance, and indistinguishability from a random oracle.

The security advantages defined below are parameterized by q , the number of the adversary’s queries to the underlying primitives (compression functions or public permutations) of hash function H . The maximum advantage is taken over all adversaries $\mathcal{A}$ restricted to q .

Collision Resistance. A hash function H is *collision-resistant* if it is computationally infeasible to find two distinct messages that hash to the same value. Formally, the advantage of an adversary $\mathcal{A}$ is:

$$\text{Adv}_H^{\text{coll}}(q) = \max_{\mathcal{A}} \Pr \left[\mathcal{A}^{H(\cdot)} \rightarrow (M, M') : M \neq M' \wedge H(M) = H(M') \right].$$

Preimage Resistance. A hash function H is *preimage-resistant* if, given a uniformly random hash value y , it is computationally infeasible to find any message that hashes to y . Formally:

$$\text{Adv}_H^{\text{pre}}(q) = \max_{\mathcal{A}} \Pr \left[y \xleftarrow{\$} \{0, 1\}^{\text{hashlen}}; \mathcal{A}^{H(\cdot)}(y) \rightarrow M : H(M) = y \right].$$

Random Multiple-Preimage Resistance (rm-pre). A hash function H is *random multiple-preimage resistant* if, given p independently and uniformly sampled

target digests, it is computationally infeasible to find a message that hashes to any one of them. The targets are externally imposed and statistically unstructured; this is the multi-target analogue of preimage resistance, and the adversary's optimal strategy is typically to concentrate its budget on the most favorable single target. Formally:

$$\text{Adv}_H^{\text{rm-pre}}(p, q) = \max_{\mathcal{A}} \Pr \left[y_1, \dots, y_p \xleftarrow{\$} \{0, 1\}^{\text{hashlen}}; \right. \\ \left. \mathcal{A}^{H(\cdot)}(y_1, \dots, y_p) \rightarrow M : \exists i \in [1, p], H(M) = y_i \right].$$

Chosen Multiple-Preimage Resistance (cm-pre). A hash function H is *chosen multiple-preimage resistant* if the same infeasibility holds when the adversary is additionally allowed to *choose* the p target digests adaptively, possibly endowing them with algebraic structure (e.g., shared prefixes or subspace-confined coordinates). Chosen-target attacks are strictly stronger: structural correlations among the targets can be exploited to amortize a single primitive query across many targets simultaneously. This notion is required whenever a security reduction exposes the target values to adversarial influence—as is the case for the orphan-target set in the Decohash simulator. Formally:

$$\text{Adv}_H^{\text{cm-pre}}(p, q) = \max_{\mathcal{A}} \Pr \left[\mathcal{A} \rightarrow y_1, \dots, y_p \in \{0, 1\}^{\text{hashlen}}; \right. \\ \left. \mathcal{A}^{H(\cdot)}(y_1, \dots, y_p) \rightarrow M : \exists i \in [1, p], H(M) = y_i \right].$$

Second-Preimage Resistance. A hash function H is *second-preimage-resistant* if, given a challenge message M sampled uniformly from the finite message space $\{0, 1\}^{\leq \ell}$ (with ℓ a prescribed length bound), it is computationally infeasible to find a different message M' with the same hash value. Formally:

$$\text{Adv}_H^{\text{2nd-pre}}(q) = \max_{\mathcal{A}} \Pr \left[M \xleftarrow{\$} \{0, 1\}^{\leq \ell}; \mathcal{A}^{H(\cdot)}(M) \rightarrow M' : M \neq M' \wedge H(M) = H(M') \right].$$

Indifferentiability. Indifferentiability [17,8] provides a stronger security guarantee: a hash construction H^f built from an ideal primitive f should be indistinguishable from a random oracle $\mathcal{R}$, even when the distinguisher has oracle access to the underlying primitive f . This notion is formalized through the two-world paradigm:

- *Real World:* The distinguisher interacts with both the hash construction H^f (implemented using primitive f) and the primitive f itself.
- *Ideal World:* The distinguisher interacts with a random oracle $\mathcal{R}$ and a simulator $\mathcal{S}$ that simulates the primitive f while maintaining consistency with $\mathcal{R}$.

The hash construction H^f is *indifferentiable* from a random oracle if there exists an efficient simulator $\mathcal{S}$ such that no distinguisher $\mathcal{D}$ —of arbitrary computational power, limited only by its query budget—can distinguish between the two worlds. Formally, the indifferentiability advantage is:

$$\text{Adv}_{(H^f, f)}^{\text{indiff}}(q_{\mathcal{S}}, q_{\mathcal{R}}) = \max_{\mathcal{D}} \left| \Pr[\mathcal{D}^{H^f, f} = 1] - \Pr[\mathcal{D}^{\mathcal{R}, \mathcal{S}^{\mathcal{R}}} = 1] \right|$$

where the distinguisher makes at most $q_{\mathcal{S}}$ queries to $\mathcal{S}$ and at most $q_{\mathcal{R}}$ queries to $\mathcal{R}$. All probabilities are taken over the random choice of the ideal primitive and the internal coins of the simulator. This notion ensures that the hash construction can securely replace a random oracle in any cryptographic scheme.

C.2 CTR-Perm and CTR-Func

Motivation Rocket-JH requires pairwise distinct round permutations for each compression and finalization round. Instantiating these as independent primitives is prohibitively expensive, while deriving variants via round constants offers no rigorous guarantee of statistical independence across exponentially many instances.

CTR-Perm and CTR-Func resolve this through domain separation: they derive 2^w effectively independent function instances from a single b -bit permutation P by indexing the input space with a w -bit counter ctr . The effective payload width is $n = b - w$. Here, *effectively independent* means that for any distinguisher making $q \ll 2^{b/2}$ queries, the family $\{P_{\text{ctr}}\}$ is indistinguishable from a family of independent random functions; each instance is a random function rather than a permutation, since the n -bit truncation destroys bijectivity.

CTR-Perm The b -bit input block is partitioned into an n -bit message x and a w -bit counter ctr . Evaluating P on $x \parallel \text{ctr}$ and retaining the leftmost n bits yields:

$$P_{\text{ctr}}(x) = \text{LMB}(n, P(x \parallel \text{ctr})).$$

Forward queries. Under the ideal-permutation assumption with $q \ll 2^{b/2}$, evaluations on distinct inputs $x \parallel \text{ctr}$ are indistinguishable from uniform. Thus, for any fixed ctr , the mapping $x \mapsto P_{\text{ctr}}(x)$ behaves as an independent random function.

Backward queries. Since P is publicly invertible, an adversary may query P^{-1} . Given (ctr, y) , the adversary searches for $y' \in \{0, 1\}^w$ such that the rightmost w bits of $P^{-1}(y \parallel y')$ equal ctr . The w -bit suffix is uniformly distributed, so each candidate satisfies the condition with probability 2^{-w} ; the number of valid preimages U follows $\text{Bin}(2^w, 2^{-w})$ with mean 1.

CTR-Func To eliminate backward-query vulnerabilities while retaining implementation efficiency, we augment CTR-Perm with a feedforward XOR:

$$P'_{\text{ctr}}(x) = \text{LMB}(n, P(x \parallel \text{ctr})) \oplus x.$$

Forward queries. Within the birthday bound $q \ll 2^{b/2}$, the term $P(x \parallel \text{ctr})$ is indistinguishable from uniform; adding x preserves this randomness. Distinct counters yield independent function instances.

Backward queries. Inverting P'_{ctr} requires solving $y = P_{\text{ctr}}(x) \oplus x$ for x given y and ctr . Rewriting, the adversary must find x and $z_R \in \{0, 1\}^w$ satisfying

$$P(x \parallel \text{ctr}) = (x \oplus y) \parallel z_R.$$

Without knowledge of P 's internal structure, the adversary must enumerate all $2^n = 2^{b-w}$ candidates for x and verify each by a forward query to P . Both forward and inverse search thus require $\Theta(2^{b-w})$ operations, which is infeasible under $q \ll 2^{b/2}$. Backward queries are effectively disabled, and CTR-Func behaves as a pure random-function family.

C.3 Decohash Framework

C.3.1 Generic Composition Let $P_0, P_1, \dots, P_z$ be independent random permutations over $\{0, 1\}^n$, and let $H[P_0]: \{0, 1\}^* \rightarrow \{0, 1\}^n$ be an *extractable* hash construction based on P_0 . For any message M , denote its output by $x_{0,1} \parallel x_{0,2} = H[P_0](M)$. The finalization function $F[P_1, \dots, P_z]: \{0, 1\}^n \rightarrow \{0, 1\}^{zh}$ is defined as

$$F(x_{0,1} \parallel x_{0,2}) = x_{1,1} \parallel x_{2,1} \parallel \dots \parallel x_{z,1},$$

where $x_{i,1} \parallel x_{i,2} = P_i(x_{i-1,1} \parallel x_{i-1,2})$ for $i = 1, \dots, z$, with $|x_{i,1}| = h \leq n/2$ and $|x_{i,2}| = n - h$. The composite Decohash is defined as $F \circ H: \{0, 1\}^* \rightarrow \{0, 1\}^{zh}$. Figure 5 depicts this construction, abstracting the three-phase Rocket-JH structure (Fig. 2) into the generic Decohash framework.

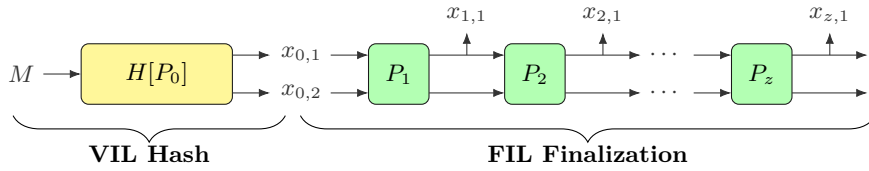


Fig. 5. Decohash Framework: VIL-Hash and FIL-Finalization Architecture

Public Permutations and Extractability A distinctive feature of Decohash is that all primitives $P_0, \dots, P_z$ are *public*: the distinguisher issues forward and backward queries to any P_i , answered by P_i itself in the real world and by $\mathcal{S}_i$ in the ideal world, and the simulator observes the complete query history. This grants *extractability*—because H and F are deterministic constructions over public permutations, any query to the high-level construction can be recognized and linked back to its origin by scanning the transcript of the underlying P_i ’s. The simulator can therefore trace the complete evaluation trace from a message M to the final output, linking it to the random oracle response $\mathcal{R}(M)$. This eliminates the “query unlinkability” problem present in traditional indistinguishability scenarios where the internal hash computation is opaque to the simulator.

Formally, H is *extractable* if there exists an efficient extractor that, given the public query transcript of P_0 and any n -bit value y , either outputs a message M with $H[P_0](M) = y$ or correctly declares y non-representable.

Indifferentiability Proof To prove that Decohash is indistinguishable from a random oracle, we exhibit a simulator $\mathcal{S} = \{\mathcal{S}_0, \mathcal{S}_1, \dots, \mathcal{S}_z\}$ such that no distinguisher $\mathcal{D}$ issuing at most q queries can tell apart the following two worlds:

- **Real World:** $\mathcal{D}$ interacts with $F \circ H$ (built from $P_0, \dots, P_z$) and the primitives themselves.
- **Ideal World:** $\mathcal{D}$ interacts with a random oracle $\mathcal{R}: \{0, 1\}^* \rightarrow \{0, 1\}^{zh}$ and $\mathcal{S}$, where each $\mathcal{S}_i$ emulates P_i consistently with $\mathcal{R}$.

The Simulator $\mathcal{S}$ The simulator maintains a transcript of input–output pairs for each emulated permutation, a set $\text{Bad}[x_0]$ of n -bit strings that have appeared as inputs to P_1 (or outputs from P_1^{-1}) but were never produced by $H[P_0]$, and internal state chains $\mathcal{C}_M$ for messages whose finalization has been initiated. The key operational principles are:

- **VIL simulation ($\mathcal{S}_0$).** $\mathcal{S}_0$ behaves as a random permutation. After each response, it recomputes all derivable VIL states $v_M = H[P_0](M)$ by tracing the public query history. If a collision $v_M = v_{M'}$ for $M \neq M'$ is discovered, event **coll** is triggered; if some v_M falls into $\text{Bad}[x_0]$, event **m-pre** is triggered. Either event causes abortion. Note that $\text{Bad}[x_0]$ is generated by the adversary and may carry algebraic structure, so **m-pre** is a *chosen*-multiple-preimage (cm-pre) event by construction.
- **FIL simulation ($\mathcal{S}_1$).** On a forward query x_0 : if $x_0 = v_M$ for some M , $\mathcal{S}_1$ retrieves $(r_1, \dots, r_z) = \mathcal{R}(M)$ and lazily constructs the chain $\mathcal{C}_M$, choosing $x_{1,2}$ so that $x_1 = r_1 \parallel x_{1,2}$ is fresh. If x_1 already belongs to $\text{Dom}(P_2)$, event **MatchDom** is triggered. If x_0 is not a valid VIL output, a random fresh output is returned and x_0 is recorded in $\text{Bad}[x_0]$. On a backward query $(x_{1,1}, x_{1,2})$: a random fresh x_0 is returned; if it coincides with a previous VIL output, event **mitm₁** occurs.
- **FIL simulation ($\mathcal{S}_i, i \geq 2$).** On a forward query: if the input extends an existing chain $\mathcal{C}_M$, $\mathcal{S}_i$ fixes $x_{i,1} = r_i$ and selects fresh $x_{i,2}$; if the resulting x_i already lies in $\text{Dom}(P_{i+1})$, event **MatchDom** is triggered. On a backward query: if the returned value equals the $(i-1)$ -st state of some chain, event **mitm_i** occurs; if it falls into $\text{Range}(P_{i-1})$, event **MatchRange** is triggered.

The events **MatchDom** and **MatchRange** capture the risk that the adversary creates shortcuts between consecutive FIL rounds by querying permutations out of order. Because the simulator samples fresh n -bit values uniformly from the unqueried space, each such collision occurs with probability at most $q/2^n$.

Indifferentiability Advantage Let E be the event that none of the bad events occur. Conditioned on E , the simulator's responses are perfectly consistent with the real world. Applying the union bound over all failure events yields:

$$\begin{aligned} \text{Adv}_{\text{Decohash}}^{\text{indiff}}(q_{\mathcal{S}_i}, q_{\mathcal{R}}) &\leq \Pr[\neg E] \\ &\leq \text{Adv}_H^{\text{coll}}(q_0) + \text{Adv}_H^{\text{cm-pre}}(p, q_0) + \frac{Q_0 q}{2^n} + \frac{3(z-1)q^2}{2^n}, \end{aligned}$$

where Q_0 is the number of distinct H -outputs after q_0 queries to P_0 and P_0^{-1} , and $q_{\mathcal{S}_i} \leq q \ll \min\{2^{n-h}, 2^{n/2}\}$.

Theorem 1. *Let $H[P_0]$ be a collision-resistant and chosen multiple preimage resistant (cm-pre) extractable hash, and $F[P_1, \dots, P_z]$ the truncating finalization defined above. Then $F \circ H: \{0, 1\}^* \rightarrow \{0, 1\}^{zh}$ is indifferentiable from a random oracle with advantage bounded by*

$$\text{Adv}_{\text{Decohash}}^{\text{indiff}}(q_{\mathcal{S}_i}, q_{\mathcal{R}}) \leq \text{Adv}_H^{\text{coll}}(q_0) + \text{Adv}_H^{\text{cm-pre}}(p, q_0) + \frac{Q_0 q}{2^n} + \frac{3(z-1)q^2}{2^n}.$$

where q_0 denotes the number of queries to P_0 and P_0^{-1} , $p \leq q$ is the number of targets in the cm-pre game, and $Q_0 \leq q_0$ is the number of distinct H -outputs—an adversary- and oracle-dependent quantity, for whose every realization the bound holds.

The Decohash framework can be viewed as a *domain extender* for a fixed-input-length random oracle, analogous to the preimage-awareness (PrA) paradigm of Dodis, Ristenpart, and Shrimpton [11]. By relaxing the security requirements of the iterative component to collision and multiple-preimage resistance, and leveraging an independent randomized finalization, efficient hash constructions can achieve indistinguishability while optimizing the VIL phase for speed.

C.3.2 Decohash on CTR-Perm Instantiating Decohash with $z+1$ independent permutations is prohibitively expensive. We resolve this via CTR-Perm, deriving all round functions from a *single* public permutation $P: \{0,1\}^b \rightarrow \{0,1\}^b$ with $b = n + w$. Each round is assigned a counter $\text{ctr} = \text{LaneID} \parallel \text{RoundCounter}$, with disjoint LaneID spaces separating the VIL and FIL phases. The effective round function is

$$P_{\text{ctr}}(x) = \text{LMB}(n, P(x \parallel \text{ctr})), \quad n = b - w.$$

Because P and P^{-1} are publicly accessible, the distinguisher may issue both forward and backward queries to any domain-separated lane. The extractability property is preserved: the simulator can trace the deterministic evaluation path of H on any message M by examining the transcript of P -queries belonging to VIL lanes, thereby extracting the internal state v_M directly.

Indifferentiability Model Let $\mathcal{H}[\text{CTR-Perm}[P]] = F \circ H$ denote the composite construction. We prove indifferentiability from a random oracle $\mathcal{R}: \{0,1\}^* \rightarrow \{0,1\}^{zh}$ in the public-permutation model:

- **Real World:** $\mathcal{D}$ has access to $(\mathcal{H}, P, P^{-1})$.
- **Ideal World:** $\mathcal{D}$ has access to $(\mathcal{R}, \mathcal{S}, \mathcal{S}^{-1})$, where $\mathcal{S}$ emulates P, P^{-1} consistently with $\mathcal{R}$.

The Simulator $\mathcal{S}$ The simulator maintains a transcript $\mathcal{T}$ of all P -queries, the set $\text{Bad}[x_0]$ of orphan n -bit inputs to FIL lanes, and internal state chains $\mathcal{C}_M$. Its operation follows three rules:

- **VIL lanes.** On any query (x, ctr) with ctr in a VIL lane, $\mathcal{S}$ returns a random fresh b -bit value (consistent with $\mathcal{T}$) and then recomputes all derivable VIL states v_M . Events *coll* and *m-pre* are checked as in the generic case.
- **FIL forward queries.** For round $i = 1$: if the n -bit input x equals a VIL output v_M , the simulator retrieves $\mathcal{R}(M) = r_1 \parallel \dots \parallel r_z$ and samples a fresh $v = r_1 \parallel \sigma \notin \text{range}(\mathcal{T})$. If the truncated payload concatenated with the next-round counter has already been queried, event *MatchDom* is triggered. If x is not a VIL output, a random fresh value is returned and x is added to $\text{Bad}[x_0]$. For rounds $i \geq 2$: if the input extends a chain $\mathcal{C}_M$, the simulator fixes $x_{i,1} = r_i$ and samples fresh $x_{i,2}$; otherwise, it returns a random consistent value.
- **FIL backward queries.** On $P^{-1}(v)$, the simulator returns a random fresh input $u = x \parallel \text{ctr} \notin \text{dom}(\mathcal{T})$, where $\text{ctr} = \text{LaneID}_{\text{fil}} \parallel 1$. If the decoded x equals a VIL output (for $i = 1$) or a chain intermediate state (for $i \geq 2$), events *mitm_i* are triggered. If x falls into $\text{Range}(P_{i-1})$, event *MatchRange* is triggered.

Bad Events and Advantage The failure events are defined analogously to the generic case, with the effective width $n = b - w$ replacing the primitive width n :

- **coll** and **m-pre**: bounded by the collision and chosen multiple-preimage (cm-pre) resistance of H .
- **mitm₁**: probability at most $Q_0 q / 2^{b-w}$.
- **mitm_i** ($i \geq 2$), **MatchDom**, **MatchRange**: each bounded by $(z - 1)q^2 / 2^{b-w}$.
- **FailFwd**: requires all 2^{b-h} values sharing prefix r_i to be occupied; impossible since $q \ll 2^{b-h}$.

Conditioned on the absence of these events, the simulator is perfectly consistent. By the union bound:

$$\mathbf{Adv}_{\mathcal{H}}^{\text{indiff}}(q_S, q_R) \leq \mathbf{Adv}_H^{\text{coll}}(q_0) + \mathbf{Adv}_H^{\text{cm-pre}}(p, q_0) + \frac{Q_0 q}{2^{b-w}} + \frac{3(z - 1)q^2}{2^{b-w}}.$$

Theorem 2 (Indifferentiability of Decohash with CTR-Perm). *Let $H[\text{CTR-Perm}[P]]$ be a collision-resistant, chosen-multiple-preimage-resistant (cm-pre), and extractable VIL hash, and let F be the FIL finalization likewise instantiated via CTR-Perm with effective width $n = b - w$. Then $\mathcal{H} = F \circ H$ is indifferentiable from a random oracle with advantage*

$$\mathbf{Adv}_{\mathcal{H}}^{\text{indiff}}(q_S, q_R) \leq \mathbf{Adv}_H^{\text{coll}}(q_0) + \mathbf{Adv}_H^{\text{cm-pre}}(p, q_0) + \frac{Q_0 q}{2^{b-w}} + \frac{3(z - 1)q^2}{2^{b-w}},$$

where $q_S \leq q \ll \min\{2^{b-h}, 2^{(b-w)/2}\}$, Q_0 is the number of distinct VIL outputs after q_0 queries (adversary- and oracle-dependent; the bound holds pointwise for every realization), and $p \leq q$.

Theorem 2 confirms that the modular security decomposition of Decohash is fully preserved under CTR-Perm. The bound mirrors that of Theorem 1 with the primitive width replaced by the effective width $b - w$. For practical parameters such as $b = 1536$, $w = 64$, and $h \leq (b - w)/2$, the negligible terms vanish under the standard assumption $q \ll 2^{b-h}$, justifying the use of a single large permutation as a cost-efficient realization of the pairwise-independent round functions required by the Decohash framework.

C.3.3 Decohash on CTR-Func

Overview While CTR-Perm achieves efficient domain separation, its susceptibility to backward queries necessitates conservative parameters and complicates the security bound. CTR-Func eliminates this limitation by augmenting the permutation output with a feedforward XOR, yielding a one-way function family that effectively disables backward queries. Under the restriction $z = 1$ and $h = n$, the FIL finalization collapses to a single invocation of CTR-Func, producing the full n -bit output without truncation. The composite construction simplifies to

$$\mathcal{H}(M) = P'_{\text{ctr}}(H[P_0](M)), \quad P'_{\text{ctr}}(x) = \text{LMB}(n, P(x \parallel \text{ctr})) \oplus x,$$

where P'_{ctr} is the sole CTR-Func instance for finalization and $n = b - w$.

Indifferentiability Model Both the adversary and the simulator have complete access to the public permutation P and its inverse P^{-1} . However, because CTR-Func computes

$$P'_{\text{ctr}}(x) = \text{LMB}(n, P(x \parallel \text{ctr})) \oplus x,$$

inverting a given output y requires solving $y = \text{LMB}(n, P(x \parallel \text{ctr})) \oplus x$ for x . Without knowledge of P 's internal structure, this demands $\Theta(2^n)$ operations in the generic group model. Consequently, the adversary's effective capability through backward queries is reduced to the birthday bound of the underlying b -bit permutation.

The Simulator The simulator maintains the transcript $\mathcal{T}$ of all P -queries, the orphan set $\text{Bad}[x_0]$, and the set of derivable VIL states. Because $z = 1$, there are no intermediate FIL chains; the simulator relates the single FIL output directly to the random oracle response.

- **VIL lanes.** Identical to the CTR-Perm case: the simulator answers with fresh random values, recomputes all derivable VIL states v_M by tracing the public transcript, and checks for events **coll** and **m-pre**.
- **FIL forward query.** On input x_0 , if $x_0 = v_M$ for some message M , the simulator retrieves $\mathcal{R}(M)$ and must sample a fresh b -bit output $v \notin \text{range}(\mathcal{T})$ satisfying

$$\text{LMB}(n, v) \oplus x_0 = \mathcal{R}(M).$$

Since $h = n$, the entire n -bit payload is fixed by this equation, leaving exactly 2^w degrees of freedom in the truncated w -bit suffix. The event **FailFwd** occurs only if all 2^w such values are already occupied in $\text{range}(\mathcal{T})$. Under the theorem assumption $q \leq 2^{n/2}$ (where $n = b - w$), the probability of this exhaustion is bounded by $q/2^{n/2}$: when $q \ll 2^w$ the probability is 0, and when $2^w \leq q \leq 2^{n/2}$ the birthday bound $q/2^{n/2}$ dominates. If x_0 is not a VIL output, a random fresh value is returned and x_0 is recorded in $\text{Bad}[x_0]$.

- **FIL backward query.** On $P^{-1}(v)$, the simulator returns a random fresh input $u = x \parallel \text{ctr} \notin \text{dom}(\mathcal{T})$, where $\text{ctr} = \text{LanID}_{\text{fil}} \parallel 1$. If the decoded x equals a VIL output v_M , the adversary learns the pair (x, v) such that $P(v_M \parallel \text{ctr}) = v$. In the real world, the FIL value would be

$$\text{LMB}(n, v) \oplus v_M = \mathcal{H}(M);$$

in the ideal world, $\mathcal{H}(M) = \mathcal{R}(M)$. Since $\mathcal{R}(M)$ is uniformly random and independent of the adversary's choice of v , the equality $\text{LMB}(n, v) \oplus v_M = \mathcal{R}(M)$ holds with probability exactly 2^{-n} . Across q queries to at most $Q_0 \leq q$ distinct VIL outputs, the probability that any backward query creates such an inconsistency is bounded by $q^2/2^n \leq q/2^{n/2}$ (using $q \leq 2^{n/2}$).

Bad Events and Advantage With $z = 1$ and $h = n$, the multi-round failure events of the generic Decohash framework vanish:

- **MatchDom** and **MatchRange** are impossible because there is only one FIL round and no adjacent rounds to bridge.
- **mitm_i** for $i \geq 2$ do not exist.

- mitm_1 is neutralized by the feedforward XOR. In CTR-Perm, a backward query returning $x = v_M$ immediately yields the FIL output via P^{-1} . In CTR-Func, the adversary obtains only the raw permutation pair (v_M, v) ; the actual FIL value $\text{LMB}(n, v) \oplus v_M$ equals the random oracle response $\mathcal{R}(M)$ with probability 2^{-n} . The adversary cannot verify this equality without querying $\mathcal{R}(M)$, and even then the chance of a successful cross-query linkage is bounded by the birthday term $q/2^{n/2}$.
- FailFwd is bounded by $q/2^{n/2}$ as argued above.

The only remaining significant failure events are coll and m-pre , inherited from the VIL component. Conditioning on their absence, the ideal and real worlds are perfectly consistent. The distinguishing advantage is therefore bounded by the VIL security plus the negligible birthday term:

$$\text{Adv}_{\mathcal{H}}^{\text{indiff}}(q_S, q_R) \leq \text{Adv}_H^{\text{coll}}(q_0) + \text{Adv}_H^{\text{cm-pre}}(p, q_0) + \frac{3q}{2^{n/2}},$$

where $n = b - w$ and $q_S \leq q \leq 2^{n/2}$.

Theorem 3 (Indifferentiability of Decohash with CTR-Func). *Let $H[\text{CTR-Func}[P]]$ be a collision-resistant, chosen-multiple-preimage-resistant (cm-pre), and extractable VIL hash, and let the FIL finalization be instantiated by a single CTR-Func invocation with $z = 1$ and $h = n = b - w$. Then the composite construction $\mathcal{H}$ is indifferentiable from a random oracle with advantage*

$$\text{Adv}_{\mathcal{H}}^{\text{indiff}}(q_S, q_R) \leq \text{Adv}_H^{\text{coll}}(q_0) + \text{Adv}_H^{\text{cm-pre}}(p, q_0) + \frac{3q}{2^{n/2}},$$

where $q_S \leq q \leq 2^{n/2}$, Q_0 is the number of distinct VIL outputs after q_0 queries, and $p \leq q$.

Compared with Theorem 2, the CTR-Func instantiation eliminates the $Q_0 q/2^n$ mitm term and the $(z - 1)q^2/2^n$ cross-round collision terms, yielding a substantially tighter bound. The sole remaining overhead is the birthday term $3q/2^{n/2}$, which is inherent to any public-permutation construction and vanishes well before the $2^{n/2}$ query limit. This confirms that CTR-Func provides both structural decoupling and enhanced backward-query resistance, making it the preferred mode for high-security deployments such as C-Hash-1024.

C.4 Indifferentiability of Rocket-JH on CTR-Perm

C.4.1 R1 on Ideal Random Permutations

Internal State Propagation Let R1 denote the Rocket-JH construction excluding the hash-block generation phase:

$$\text{R1}[P_{i,1}](M) = h_{i,1} \parallel h_{i,2}, \quad i \in [0, l].$$

Since the permutations $P_{i,1}$ are public, the adversary may issue arbitrary forward and backward queries. Let q_i^+ , q_i^- , and $q_i = q_i^+ + q_i^-$ denote the forward, backward, and total query counts in round i , respectively, and let $q = \max_i q_i$.

In round $i \in [1, l]$, suppose the adversary has obtained q internal states $h_{i-1,1} \parallel h_{i-1,2}$. Let t be the maximum multiplicity of any colliding half-state value $h_{i-1,2}$ among these q states, and let $h_{i-1,2}^*$ be a value occurring t times with corresponding distinct half-states $h_{i-1,1}^{(1)}, \dots, h_{i-1,1}^{(t)}$. By querying $P_{i,1}$ at $x_{i,1}^* \parallel h_{i-1,2}^*$ for a chosen $x_{i,1}^*$, the adversary receives $y_{i,1}^* \parallel y_{i,2}^*$ and constructs up to t distinct input-output tuples

$$\left(h_{i-1,1}^{(j)}, h_{i-1,2}^*, m_i^{(j)}, h_{i,1}^*, h_{i,2}^{(j)} \right)_{j \in [1, t]}, \quad m_i^{(j)} = h_{i-1,1}^{(j)} \oplus x_{i,1}^*, \quad h_{i,1}^* = y_{i,1}^*, \quad h_{i,2}^{(j)} = y_{i,2}^* \oplus m_i^{(j)}.$$

The resulting states are pairwise distinct. Thus a single forward query to $P_{i,1}$ can yield up to t new internal states, and over $q_i^+ \leq q$ forward queries the adversary acquires at most tq distinct states at round i .

The Parameter t The parameter t is the maximum collision multiplicity for half-state values among q internal states. To ensure that $(t+1)$ -collisions remain statistically negligible, we require

$$\binom{q}{t+1} \left(\frac{1}{2^{n/2}} \right)^t < 1.$$

For the query regime $q \leq 2^{3n/8}$ used in subsequent analysis, this condition yields $t \leq 3$.

Preimage Resistance Given a challenge (h_1, h_2) , the adversary must link one of k valid initial states to the target through R1. Three strategies exist:

1. **Forward-only.** Propagating from an initial state toward the target, the adversary obtains at most tq candidate states per round. The success probability is bounded by

$$\Pr[\text{forward reach}] \leq \sum_{i=1}^l \frac{tq}{2^n} = \frac{tql}{2^n}.$$

2. **Backward-only.** Working backward from the target, the adversary must match one of k initial states, giving

$$\Pr[\text{backward reach}] \leq \frac{ktq}{2^n}.$$

3. **Meet-in-the-middle.** Combining forward and backward queries at an intermediate round i , with at most tq_i^+ and tq_{i+1}^- states respectively, the collision probability at round i is at most $t^2q^2/2^n$. Summing over l rounds:

$$\Pr[\text{mitm}] \leq \sum_{i=1}^l \frac{t^2q^2}{2^n} = \frac{t^2q^2l}{2^n}.$$

The meet-in-the-middle strategy dominates, yielding

$$\mathbf{Adv}_{\text{R1}}^{\text{pre}}(q) \leq \frac{t^2q^2l}{2^n}.$$

Random Multiple-Preimage Resistance (rm-pre) Given $p \leq q$ independently and uniformly sampled challenges, the adversary attempts to find M such that $\exists j \in [1, p]$ with $R1(M) = h_1^{(j)} \parallel h_2^{(j)}$. Repeating the preimage attack p times gives the naive bound $p \cdot \text{Adv}_{R1}^{\text{pre}}(q)$. However, for the forward-only strategy the adversary can target any of the p challenges, giving $\frac{tpql}{2^n} \leq \frac{tq^2l}{2^n}$. For backward and meet-in-the-middle strategies, the targets are statistically unstructured, so the optimal attack focuses queries on a single target and the advantage does not increase significantly with p . Thus

$$\text{Adv}_{R1}^{\text{rm-pre}}(p, q) \leq \frac{9q^2l}{2^n}, \quad \text{for } p \leq q, q \leq 2^{3n/8}, t = 3.$$

Chosen Multiple-Preimage Resistance (cm-pre) The Decohash reduction (Theorems 1 and 2) requires the VIL component to resist *chosen*-target multiple-preimage attacks, since the orphan-target set $\text{Bad}[x_0]$ in the simulator is generated by the adversary and may carry algebraic structure. For this stronger notion, the analysis of R1 changes qualitatively: structured targets allow a single backward query to be amortized over all targets simultaneously. We first prove an upper bound for the ideal multi-permutation instantiation, and then show that the Universal San Mai Attack (USMA) [15] matches it exactly.

Lemma 1 (cm-pre Bound of R1). *Let R1 be instantiated with pairwise independent random permutations over $\{0, 1\}^n$. Then, for $p \leq q$,*

$$\text{Adv}_{R1}^{\text{cm-pre}}(p, q) \leq \frac{O(q^3)}{2^n}.$$

Proof. A successful forgery requires a certified path of round transitions from the initial value to one of the p chosen target states. Each primitive query certifies exactly one round transition, and each backward answer is a single uniformly distributed n -bit value. Hence a backward answer completes a splice between one forward-propagated chain state and one target only if it aligns with both on the full n -bit state, which for any fixed (forward state, backward answer, target) triple happens with probability at most 2^{-n} . Union-bounding over at most q forward chain states, q backward answers, and $p \leq q$ targets yields $q^3/2^n$. Strategies without multi-target amortization (forward-only or backward-only) contribute at most $O(q^2)/2^n$ and are dominated.

The bound is tight. Li, Li, Hu, and Liu [15] show that iterative modes whose round output forwards one half of the state to the next round without remixing—precisely the JH-type linear layer of R1, in which $h_{i,1} = y_{i,1}$ is an unmixed half—admit a chosen-target meet-in-the-middle attack of advantage $\Theta(q^3/2^n)$: the adversary manufactures q target states sharing a common left half (with right halves confined to a q -element subspace), issues q backward queries, and each backward query simultaneously serves all q targets, yielding q^2 forged states that match the q forward-propagated states with probability $\Theta(q^3/2^n)$. Consequently,

$$\text{Adv}_{R1}^{\text{cm-pre}}(p, q) = \Theta\left(\frac{q^3}{2^n}\right),$$

i.e., the ideal multi-permutation instantiation is capped at *exactly* $n/3$ bits, with no gap between the provable upper bound and the best known attack.

Collision Resistance. The adversary seeks distinct messages $M \neq M'$ with $R1(M) = R1(M')$. Let $m_1 \cdots m_l$ and $m'_1 \cdots m'_l$ be the padded blocks.

Case 1: Different lengths ($l \neq l'$). Assuming $l' < l$, the state of M' after l' rounds is fixed. A collision requires connecting the state of M after l' rounds to this fixed state through the remaining $l - l'$ rounds, which reduces to a preimage attack on a shortened R1:

$$\Pr[\text{collision} \mid l \neq l'] \leq \frac{t^2 q^2 (l - l')}{2^n} < \frac{t^2 q^2 l}{2^n}.$$

Case 2: Equal lengths ($l = l'$). For round $i = 0$, the k possible initial states collide with probability at most $\binom{k}{2}/2^n$, which is negligible. For $i \in [1, l]$, conditioning on no earlier collision, a collision at round i requires

$$P_{i,1}(x_{i,1} \parallel x_{i,2}) \oplus P_{i,1}(x'_{i,1} \parallel x'_{i,2}) = 0^{n/2} \parallel (m_i \oplus m'_i),$$

with $x_{i,1} = h_{i-1,1} \oplus m_i$ and $x_{i,2} = h_{i-1,2}$. If $m_i = m'_i$, injectivity of $P_{i,1}$ forces $h_{i-1,1} \parallel h_{i-1,2} = h'_{i-1,1} \parallel h'_{i-1,2}$, contradicting the no-collision assumption. Hence $m_i \neq m'_i$ is necessary.

The adversary makes at most q queries to $P_{i,1}$ and $P_{i,1}^{-1}$ per round. We bound the collision probability by query direction to the invocations on $P_{i,1}$:

– **Forward–forward.** By the birthday bound over tq_i^+ states:

$$\Pr[\text{collision} \mid \text{FF}] \leq \binom{tq_i^+}{2} \frac{1}{2^n} \leq \frac{t^2 q^2}{2^{n+1}}.$$

– **Backward–backward.** The adversary fixes $h_{i,1}^*$, makes q_i^- inverse queries of the form $h_{i,1}^* \parallel y_{i,2}$, and matches outputs against at most tq_{i-1}^+ forward-propagated states. The probability is

$$\Pr[\text{collision} \mid \text{BB}] \leq \binom{tq_{i-1}^+}{2} \binom{q_i^-}{2} \left(\frac{1}{2^{n/2}}\right)^3 \leq \frac{t^2 q^4}{2^{3n/2+2}}.$$

– **Forward–backward (FtB).** The adversary first accumulates at most tq round- i states via forward queries into a set $\mathcal{H}_i$, then selects one target state $(h_{i,1}^*, h_{i,2}^*) \in \mathcal{H}_i$ and issues q backward queries $P_{i,1}^{-1}(h_{i,1}^*, z_{i,2}^{(j)})$. For a single backward query to link back to a legitimate prior state $(h_{i-1,1}, h_{i-1,2}) \in \mathcal{H}_{i-1}$, two independent $n/2$ -bit conditions must hold:

1. the returned right half must satisfy $x_{i,2} = h_{i-1,2}$, which occurs with probability at most $tq/2^{n/2}$ because there are tq candidate right-half values in $\mathcal{H}_{i-1}$;
2. the derived left half must satisfy $x_{i,1} \oplus z_{i,2}^{(j)} \oplus h_{i,2}^* = h_{i-1,1}$, which, since $x_{i,1}$ is uniformly random, holds with probability at most $1/2^{n/2}$.

Multiplying the two independent probabilities and applying a union bound over all q backward queries yields

$$\Pr[\text{collision} \mid \text{FtB}] \leq q \cdot \frac{tq}{2^n} = \frac{tq^2}{2^n}.$$

– **Backward–forward (BtF).** The adversary first issues q backward queries $P_{i,1}^{-1}(h_{i,1}^{(j)}, z_{i,2}^{(j)})$ to obtain q candidate predecessors $(x_{i,1}^{(j)}, x_{i,2}^{(j)})$. A single candidate

yields a right-half match with the tq legitimate states in $\mathcal{H}_{i-1}$ with probability at most $tq/2^{n/2}$; conditioned on this, the adversary can freely adjust the message block m_i to satisfy the left-half alignment, so the left-half condition holds with probability 1. Thus the expected number of complete predecessor matches is bounded by $tq^2/2^{n/2}$. To turn any such match into a full collision, the adversary must then produce a second chain that hits the same round- i state via a forward query; with q forward queries this succeeds with probability at most $q/2^n$. Multiplying the three stages gives

$$\Pr[\text{collision} \mid \text{BtF}] \leq \frac{tq^2}{2^{n/2}} \cdot \frac{q}{2^n} = \frac{tq^3}{2^{3n/2}}.$$

Since $t \leq 3$ for the query regime $q \leq 2^{3n/8}$, both hybrid strategies are asymptotically dominated by the pure forward-forward and backward-backward bounds:

$$\frac{tq^2}{2^n} \leq \frac{t^2q^2}{2^{n+1}} \quad \text{and} \quad \frac{tq^3}{2^{3n/2}} \leq \frac{t^2q^4}{2^{3n/2+2}},$$

so neither FtB nor BtF improves upon the FF and BB attacks. Consequently, the overall equal-length collision probability is governed by

$$\frac{t^2q^2l}{2^{n+1}} + \frac{t^2q^4l}{2^{3n/2+2}}.$$

Overall bound. Combining both cases and substituting $t = 3$:

$$\mathbf{Adv}_{\text{R1}}^{\text{coll}}(q) \leq \frac{9q^2l}{2^n} + \frac{9q^4l}{2^{3n/2+2}}, \quad \text{for } q \leq 2^{3n/8}.$$

C.4.2 R1 on CTR-Perm When instantiating Rocket-JH with CTR-Perm, the counter is structured as

$$\text{ctr}_{i,j} = \text{LaneID}_j \parallel \text{RoundCounter}_i.$$

The VIL compression lanes and FIL finalization lanes occupy disjoint `LaneID` spaces, while each internal round employs a distinct `RoundCounter`. Consequently, every permutation call—whether for message absorption or hash-block generation—receives a unique counter value. Within $q \ll 2^{b/2}$ queries (the birthday bound of the underlying permutation), these domain-separated instances remain cryptographically isolated, effectively behaving as independent random functions.

We denote the CTR-Perm instantiation of the iterative component by R1CP. By Theorem 2, the security guarantees of R1CP follow directly from those of R1 with effective state width $n = b - w$, with one strengthening exception: the cm-pre bound improves beyond a direct transplantation of the R1 bound. Indeed, a backward lane query under CTR-Perm is not free—inverting a lane output under its prescribed counter requires an expected 2^w primitive queries, since the number of valid preimages follows $\text{Bin}(2^w, 2^{-w})$ with mean 1 (cf. the backward-query analysis of CTR-Perm above). The USMA adversary must therefore pay a factor 2^w per amortized backward query; with q structured targets and $2^n/q^2$ backward inversions

the success probability reaches $\Theta(1)$ at total cost $q + (2^n/q^2) \cdot 2^w$, minimized at $q \approx 2^{(n+w)/3}$. The cm-pre advantage in the total query count is thus $O(q^3)/2^b$, corresponding to a $b/3$ -bit ceiling rather than $n/3$. Specifically, for $p \leq q$ and $q \leq 2^{3(b-w)/8}$:

$$\mathbf{Adv}_{\text{R1CP}}^{\text{rm-pre}}(p, q) \leq \frac{9q^2l}{2^{b-w}}, \quad \mathbf{Adv}_{\text{R1CP}}^{\text{cm-pre}}(p, q) \leq \frac{O(q^3)}{2^b}, \quad \mathbf{Adv}_{\text{R1CP}}^{\text{coll}}(q) \leq \frac{9q^2l}{2^{b-w}} + \frac{9q^4l}{2^{3(b-w)/2+2}}.$$

C.4.3 Indifferentiability of Rocket-1

Definition Rocket-1 denotes the full Rocket-JH construction instantiated via CTR-Perm: the VIL compression phase employs domain-separated CTR-Perm instances $P_{i, \text{ID}_{\text{vil}}}$ for $i \in [0, l]$, and the FIL finalization phase employs $P_{i, \text{ID}_{\text{fil}}}$ for $i \in [1, z]$, all derived from a single public permutation $P: \{0, 1\}^b \rightarrow \{0, 1\}^b$ with effective width $n = b - w$. The iterative component R1 realized under this instantiation is denoted R1CP.

Security bound Applying Theorem 2 with $H = \text{R1CP}$ and $h = (b - w)/2$, the FIL-phase mitm_i terms are bounded by $q^2/2^{b-w}$ for $i = 1$ and $(z - 1)q^2/2^{b-w}$ for $i \geq 2$, while **MatchDom** and **MatchRange** contribute at most $2(z - 1)q^2/2^{b-w}$. The VIL component additionally contributes its rm-pre and cm-pre advantages; the cm-pre term $O(q^3/2^b)$, imposed by the Universal San Mai Attack [15] and raised by the 2^w lane-inversion cost of backward queries (see the R1CP analysis above), dominates all FIL-phase terms and, for C-Hash-512, the entire bound. The **FailFwd** event is negligible because $q \ll 2^{b-h}$. The full indifferentiability advantage is therefore

$$\mathbf{Adv}_{\text{Rocket-1}}^{\text{indiff}}(q_{\mathcal{S}_i}, q_{\mathcal{R}}) \leq \frac{19q^2l}{2^{b-w}} + \frac{3(z - 1)q^2}{2^{b-w}} + \frac{9q^4l}{2^{3(b-w)/2+2}} + \frac{O(q^3)}{2^b}.$$

The cm-pre term caps the indifferentiability of Rocket-1 at $b/3 = 512$ bits; the quartic and quadratic terms become active only at the higher ceilings $3(b - w)/8$ and $(b - w - \log_2 l)/2$, respectively. For $l \geq z$, this matches—and at the dominant cm-pre term exceeds—the asymptotic security profile of the ideal multi-permutation instantiation, confirming that CTR-Perm achieves practical domain separation without compromising the analytical benefits of structural decoupling.

C.5 Indifferentiability of Rocket-JH on CTR-Func

C.5.1 R1 on CTR-Func: Analysis without the Parameter t

Observation (Single-Round Damping). In CTR-Func, a forward query on t colliding right-half states $h_{i-1,2}^*$ produces t new states whose right halves are determined as follows. Let $P_i(x_{i,1} \parallel x_{i,2}) = z_{i,1} \parallel z_{i,2}$ denote the n -bit output of the i -th round permutation, split into left and right halves of $n/2$ bits each. Then

$$h_{i,2}^{(j)} = z_{i,2}^* \oplus h_{i-1,2}^* \oplus h_{i-1,1}^{(j)} \oplus x_{i,1}^*, \quad j \in [1, t].$$

Because the predecessor left halves $h_{i-1,1}^{(j)}$ are pairwise distinct, the resulting $h_{i,2}^{(j)}$ are *uniformly random and pairwise distinct*. Consequently, these t states *cannot*

share a common right half in the next round; the multiplicity advantage vanishes after a single round.

Similarly, each backward query yields a valid predecessor with probability $2^{-n/2}$, and the returned right half $x_{i,2}$ is uniformly random. Hence backward queries cannot be multiplexed across colliding right halves either.

This *single-round damping* property implies that t -multiplicity acts as a one-shot coupon: it may save $t - 1$ queries in one round, but the very next round requires t separate queries to advance those t states. Amortized over any chain of length ≥ 2 , the net gain is at most a constant factor. Therefore, for the purpose of asymptotic bounds, we treat the number of distinct states that the adversary can prepare at any fixed round as $O(q)$, absorbing the t -factor into the hidden constant.

The following analysis removes the explicit parameter t by exploiting this damping property. It yields per-attribute query limits that differ according to the underlying information-theoretic barrier.

Forward queries. Let q denote the maximum number of queries (forward or backward) that the adversary makes to the public permutation P in any single round. Over l rounds the total query budget is at most ql . Because of single-round damping, each forward query produces one uniformly random n -bit state, except when it exploits a right-half collision of multiplicity t , in which case it produces t states whose right halves are pairwise distinct. Advancing those t states to the next round, however, requires t separate queries. Hence, with a per-round budget of q queries, the adversary can maintain at most $O(q)$ active chains simultaneously. After any number of rounds $i \geq 1$, the adversary holds at most $O(q)$ candidate states, and in particular after the final round l he holds at most $O(q)$ terminal states.

Backward queries. Given a target state $(h_{i,1}^*, h_{i,2}^*)$, the adversary chooses $z_{i,1} \parallel z_{i,2}$ and queries P^{-1} . The left half of the predecessor satisfies $h_{i-1,1} = x_{i,1} \oplus m_i$ and is uniformly random over $\{0, 1\}^{n/2}$ because $x_{i,1}$ is the random output of P^{-1} . For this predecessor to be a valid link to the target, the equation $h_{i,1}^* = z_{i,1} \oplus x_{i,1}$ must hold, which occurs with probability exactly $2^{-n/2}$. Hence, out of q backward queries in a round, at most $q/2^{n/2}$ are valid predecessors in expectation; each valid predecessor is itself a uniformly random n -bit state.

Preimage Resistance. Given a challenge (h_1, h_2) , the adversary must link one of the k valid initial states to the target. Since the preimage game imposes no restriction on message length, the adversary may propagate through any number of rounds up to the maximum l .

1. **Forward-only.** Starting from one of the k initial states, the adversary makes at most q forward queries per round. Owing to single-round damping, each query contributes at most one new candidate state to the next round; hence after any round $i \in [1, l]$ the adversary holds at most $O(q)$ states. Because the adversary is free to choose the message length, he may explore *all* l rounds independently. Across the full l -round spectrum the total number of terminal states examined is therefore $O(ql)$, each uniformly distributed over $\{0, 1\}^n$. The probability that

any of them equals the target is at most

$$\Pr[\text{forward reach}] \leq \frac{O(ql)}{2^n}.$$

2. **Backward-only.** Working backward from the target, a single backward query produces a valid predecessor with probability $2^{-n/2}$ (the right-half $x_{i,2}$ must match). After $j \geq 1$ rounds the expected number of reachable states is at most $q/2^{jn/2} \leq q/2^{n/2}$. For each such candidate, the left-half $x_{i,1}$ is uniformly random; matching one of the initial left-half values requires another $2^{-n/2}$ factor. Thus

$$\Pr[\text{backward reach}] \leq \frac{q}{2^{n/2} \cdot 2^{n/2}} \leq \frac{O(q)}{2^n}.$$

3. **Meet-in-the-middle.** The adversary partitions the chain at an intermediate round $i \in [1, l]$. Forward propagation to round i yields $O(q)$ candidate states, while backward propagation from the target yields $O(q/2^{n/2})$ valid predecessors. A collision at round i occurs with probability at most

$$\Pr[\text{mitm at round } i] \leq \frac{O(q) \cdot (q/2^{n/2})}{2^n} = \frac{O(q^2)}{2^{3n/2}}.$$

Summing over the l possible intermediate rounds:

$$\Pr[\text{mitm}] \leq \sum_{i=1}^l \frac{O(q^2)}{2^{3n/2}} = \frac{O(q^2l)}{2^{3n/2}}.$$

The overall preimage advantage against R1 on CTR-Func (R1CF) is therefore

$$\text{Adv}_{\text{R1CF}}^{\text{pre}}(q) \leq \frac{O(ql)}{2^n} + \frac{O(q)}{2^n} + \frac{O(q^2l)}{2^{3n/2}} \leq \frac{O(ql)}{2^n} + \frac{O(q^2l)}{2^{3n/2}}.$$

Multiple-Preimage Resistance (rm-pre and cm-pre). Given $p \leq q$ challenges, the adversary seeks a message M whose hash collides with any one of the p targets.

- **Forward-only.** The forward strategy can aim at any of the p challenges. Propagating through all l rounds produces $O(ql)$ candidate states per challenge, yielding

$$\Pr[\text{forward reach}] \leq \frac{O(ql)p}{2^n} \leq \frac{O(q^2l)}{2^n}.$$

- **Meet-in-the-middle.** The MITM strategy against p targets proceeds exactly as in the single-preimage case, except the adversary may select the most advantageous target. The bound is therefore unchanged in form:

$$\Pr[\text{mitm}] \leq \frac{O(q^2l)}{2^{3n/2}}.$$

Hence

$$\text{Adv}_{\text{R1CF}}^{\text{rm-pre}}(p, q) \leq \frac{O(q^2l)}{2^n} + \frac{O(q^2l)}{2^{3n/2}}.$$

The identical bound holds for chosen-target (cm-pre) attacks: the forward-only strategy already optimizes over arbitrary target sets, while backward and

meet-in-the-middle strategies cannot amortize one backward query across structured targets, because each valid backward link costs $2^{-n/2}$ per round per target. Consequently,

$$\text{Adv}_{\text{R1CF}}^{\text{cm-pre}}(p, q) \leq \frac{O(q^2 l)}{2^n} + \frac{O(q^2 l)}{2^{3n/2}},$$

and the USMA-type amplification that limits R1 on CTR-Perm does not apply to R1CF.

Collision Resistance. The adversary seeks $M \neq M'$ with $\text{R1CF}(M) = \text{R1CF}(M')$.

Case 1: Different lengths ($l \neq l'$). Assuming $l' < l$, the state of M' after l' rounds is fixed. A collision requires finding M of length l such that its final state after l rounds equals this fixed state. This reduces to a preimage attack on a shortened R1CF with l rounds, and is bounded by the full preimage advantage:

$$\Pr[\text{collision} \mid l \neq l'] \leq \frac{O(ql)}{2^n} + \frac{O(q^2 l)}{2^{3n/2}}.$$

Case 2: Equal lengths ($l = l'$). A collision at round i requires two distinct predecessor states yielding identical outputs. We analyze by query direction:

- **Forward–forward.** Owing to single-round damping, the adversary holds at most $O(q)$ distinct states at any fixed round i . A collision at round i requires two of these states to coincide, which by the birthday bound occurs with probability at most $\binom{O(q)}{2}/2^n$. Summing over all l rounds,

$$\Pr[\text{collision} \mid \text{FF}] \leq \sum_{i=1}^l \binom{O(q)}{2} \frac{1}{2^n} \leq \frac{O(q^2 l)}{2^{n+1}}.$$

- **Backward–backward.** The adversary makes q backward queries to P_i^{-1} and, among the returned preimages, seeks two that match distinct states in $\mathcal{H}_{i-1}$ (the set of $O(q)$ forward-propagated states at round $i-1$) and yet produce the same round- i output.

Select two distinct prior states $(h_{i-1,1}^{(a)}, h_{i-1,2}^{(a)})$ and $(h_{i-1,1}^{(b)}, h_{i-1,2}^{(b)})$ from $\mathcal{H}_{i-1}$ in $\binom{O(q)}{2}$ ways, and select two distinct backward queries $j, k \in [1, q]$ in $\binom{q}{2}$ ways.

Query j (resp. k) matches state a (resp. b) if the returned right-half satisfies $x_{i,2}^{(j)} = h_{i-1,2}^{(a)}$ (resp. $x_{i,2}^{(k)} = h_{i-1,2}^{(b)}$). Because P_i is a random permutation, each right-half match occurs independently with probability $2^{-n/2}$, so both match simultaneously with probability $(2^{-n/2})^2$.

Conditioned on these matches, the corresponding message blocks are $m_i^{(j)} = h_{i-1,1}^{(a)} \oplus x_{i,1}^{(j)}$ and $m_i^{(k)} = h_{i-1,1}^{(b)} \oplus x_{i,1}^{(k)}$. Under CTR-Func the round- i outputs are

$$h_{i,1}^{(\cdot)} = z_{i,1}^{(\cdot)} \oplus x_{i,1}^{(\cdot)}, \quad h_{i,2}^{(\cdot)} = z_{i,2}^{(\cdot)} \oplus h_{i-1,2}^{(\cdot)} \oplus m_i^{(\cdot)}.$$

Because $x_{i,1}^{(j)}$ and $x_{i,1}^{(k)}$ are uniform and independent (given the distinct right-half matches), the pair $(h_{i,1}^{(j)}, h_{i,2}^{(j)})$ and $(h_{i,1}^{(k)}, h_{i,2}^{(k)})$ are independent uniform n -bit strings. Their total collision probability is therefore 2^{-n} .

Aggregating over round choices and combining the three independent factors:

$$\Pr[\text{collision} \mid \text{BB}] \leq \sum_{i=1}^l \binom{O(q)}{2} \binom{q}{2} \left(\frac{1}{2^{n/2}}\right)^2 \frac{1}{2^n} \leq \frac{O(q^4 l)}{2^{2n+1}}.$$

This is negligible compared with the FF term for all $q \ll 2^{n/2}$, confirming that the quartic backward-backward term is reduced from $O(q^4/2^{3n/2})$ to $O(q^4/2^{2n})$, rendering it negligible compared with the quadratic term for all $q \ll 2^{n/2}$.

- **Forward-backward (FtB).** The adversary first accumulates $O(q)$ round- i states via forward queries into a set $\mathcal{H}_i$, then selects one target state $(h_{i,1}^*, h_{i,2}^*) \in \mathcal{H}_i$ and issues q backward queries $P_i^{-1}(z_{i,1}^{(j)} \parallel z_{i,2}^{(j)})$.

For a single backward query j to link back to a legitimate prior state $a \in \mathcal{H}_{i-1}$, two independent events must occur:

1. *Right-half match.* The returned preimage must satisfy $x_{i,2}^{(j)} = h_{i-1,2}^{(a)}$. Since there are $O(q)$ candidates in $\mathcal{H}_{i-1}$ and the right half is uniform, this occurs with probability at most $O(q)/2^{n/2}$.
2. *Full-state match.* Conditioned on the right-half match, the left half $x_{i,1}^{(j)}$ is uniform over $\{0, 1\}^{n/2}$. Under CTR-Func, the resulting round- i state $(h_{i,1}^{(j)}, h_{i,2}^{(j)})$ is a uniform n -bit string. The probability that it equals the chosen target $(h_{i,1}^*, h_{i,2}^*)$ is exactly 2^{-n} .

Applying a union bound over the q backward queries and summing over l rounds:

$$\Pr[\text{collision} \mid \text{FtB}] \leq \sum_{i=1}^l q \cdot \frac{O(q)}{2^{n/2}} \cdot \frac{1}{2^n} = \frac{O(q^2 l)}{2^{3n/2}}.$$

- **Backward-forward (BtF).** The adversary first issues q backward queries. Each query produces a valid predecessor matching some state in $\mathcal{H}_{i-1}$ with probability $O(q)/2^{n/2}$; the expected number of complete matches is therefore $O(q^2)/2^{n/2}$. Each such match defines a specific round- i state $(h_{i,1}^*, h_{i,2}^*)$.

To turn any such match into a full collision, the adversary must then produce a *second* message path that reaches the same round- i state via a forward query. With q forward queries and a uniform n -bit target, this succeeds with probability at most $q/2^n$. Multiplying the three stages and summing over l rounds:

$$\Pr[\text{collision} \mid \text{BtF}] \leq \sum_{i=1}^l \frac{O(q^2)}{2^{n/2}} \cdot \frac{q}{2^n} = \frac{O(q^3 l)}{2^{3n/2}}.$$

For $q \leq 2^{n/2}$, this is asymptotically dominated by the FF term $O(q^2 l)/2^n$.

Summing over l rounds, the equal-length collision advantage collects contributions from all four query-direction strategies:

$$\Pr[\text{collision} \mid l = l'] \leq \underbrace{\frac{O(q^2 l)}{2^{n+1}}}_{\text{FF}} + \underbrace{\frac{O(q^4 l)}{2^{2n+1}}}_{\text{BB}} + \underbrace{\frac{O(q^2 l)}{2^{3n/2}}}_{\text{FtB}} + \underbrace{\frac{O(q^3 l)}{2^{3n/2}}}_{\text{BtF}} \leq \frac{O(q^2 l)}{2^n} + \frac{O(q^2 l)}{2^{3n/2}},$$

where the BB term is absorbed by $O(q^2 l)/2^n$ for $q \ll 2^{n/2}$, and the BtF term is absorbed by the same dominant term under $q \leq 2^{n/2}$.

Overall bound. Combining both cases, the $O(ql)/2^n$ term from the unequal-length case is absorbed by the $O(q^2l)/2^n$ term from the equal-length case (for $q \geq 1$). The overall collision advantage is therefore

$$\mathbf{Adv}_{\text{R1CF}}^{\text{coll}}(q) \leq \frac{O(q^2l)}{2^n} + \frac{O(q^2l)}{2^{3n/2}}.$$

Second-Preimage Resistance. Given a challenge message M' of length l' with hash output $h'_{i'}$, the adversary seeks $M \neq M'$ with $\text{R1CF}(M) = h'_{i'}$. Two mutually exclusive strategies arise:

Strategy 1: Direct preimage attack. Treating $h'_{i'}$ as an arbitrary target, the success probability is bounded by the full preimage advantage

$$\mathbf{Adv}_{\text{R1CF}}^{\text{pre}}(q) \leq \frac{O(ql)}{2^n} + \frac{O(q^2l)}{2^{3n/2}}.$$

Strategy 2: Equal-length intermediate collision. The adversary constructs M with $l = l'$ that collides with M' at some round $i' \in [1, l']$.

- **Forward propagation.** At round i' the adversary holds at most $O(q)$ distinct candidate states, each uniform over $\{0, 1\}^n$. The probability that one equals the fixed target $h'_{i'}$ is at most $O(q)/2^n$. Summing over all $i' \in [1, l']$:

$$\sum_{i'=1}^{l'} \frac{O(q)}{2^n} = \frac{O(ql')}{2^n}.$$

- **Backward linkage.** Querying backward from $h'_{i'}$ yields at most $q/2^{n/2}$ valid states per round; a meet-in-the-middle collision with $O(q)$ forward states at round j occurs with probability at most $O(q^2)/2^{3n/2}$. Aggregating over l' possible collision points:

$$\sum_{i'=1}^{l'} \frac{O(q^2)}{2^{3n/2}} = \frac{O(q^2l')}{2^{3n/2}}.$$

Overall bound. Combining the two strategies:

$$\mathbf{Adv}_{\text{R1CF}}^{\text{2nd-pre}}(q) \leq \frac{O(ql)}{2^n} + \frac{O(q^2l)}{2^{3n/2}} + \frac{O(ql')}{2^n} + \frac{O(q^2l')}{2^{3n/2}} \leq \frac{O(q(l+l'))}{2^n} + \frac{O(q^2(l+l'))}{2^{3n/2}}.$$

C.5.2 Indifferentiability of Rocket-1f

Definition Rocket-1f denotes the full Rocket-JH construction instantiated entirely via CTR-Func: the VIL compression lanes employ domain-separated CTR-Func instances $P'_{i, \text{D}_{\text{vil}}}$ for $i \in [0, l]$, and the FIL finalization lane employs a single CTR-Func instance $P'_{1, \text{D}_{\text{fil}}}$ with $z = 1$ and $h = n = b - w$. The underlying public permutation $P: \{0, 1\}^b \rightarrow \{0, 1\}^b$ is shared across all lanes, with effective state width $n = b - w$.

Security bound Applying Theorem 3 with $H = \text{R1f}$ (the R1 iterative component on CTR-Func) and the restriction $z = 1$, $h = n$, the indifferenciability advantage is governed by the VIL component’s collision and multiple-preimage resistance plus a negligible birthday term:

$$\mathbf{Adv}_{\text{Rocket-1f}}^{\text{indiff}}(q_S, q_R) \leq \mathbf{Adv}_{\text{R1CF}}^{\text{coll}}(q_0) + \mathbf{Adv}_{\text{R1CF}}^{\text{cm-pre}}(p, q_0) + \frac{3q}{2^{(b-w)/2}},$$

where $q_S \leq q$ and $q_R \leq q$.

By the single-round damping analysis of R1CF (Appendix C.5.1), the concrete bounds are

$$\mathbf{Adv}_{\text{R1CF}}^{\text{coll}}(q) \leq \frac{O(q^2l)}{2^{b-w}} + \frac{O(q^2l)}{2^{3(b-w)/2}}, \quad \mathbf{Adv}_{\text{R1CF}}^{\text{cm-pre}}(p, q) \leq \frac{O(q^2l)}{2^{b-w}} + \frac{O(q^2l)}{2^{3(b-w)/2}}.$$

For $q \leq 2^{(b-w)/2}$ the first term dominates in both bounds. Summing the dominant terms and absorbing the lower-order $2^{-3n/2}$ contributions, we obtain

$$\mathbf{Adv}_{\text{Rocket-1f}}^{\text{indiff}}(q_S, q_R) \leq \frac{O(q^2l)}{2^{b-w}} + \frac{3q}{2^{(b-w)/2}}.$$

For the concrete parameters $n = b - w = 1472$ and $l \leq 2^{56}$, the collision term reaches 1 when $q \approx 2^{(b-w)/2}/\sqrt{l} \approx 2^{708}$. Hence the generic indifferenciability guarantee holds for

$$q_R \leq 2^{708}, \quad q_S \leq q \leq 2^{708}.$$

Preimage and second-preimage beyond the generic bound. The generic indifferenciability reduction is limited by the birthday barrier of the n -bit internal state, which caps the collision term at $q \sim 2^{n/2}$. However, this barrier affects only collision resistance; it does not preclude stronger preimage or second-preimage bounds. A direct analysis of the VIL-FIL composition (Appendix C.8) shows that, because the FIL finalization F' is a uniform random function from $\{0, 1\}^n$ to $\{0, 1\}^{\text{hashlen}}$ with $\text{hashlen} = 1024 < n$, the overall construction satisfies

$$\mathbf{Adv}_{\text{C-Hash-1024}}^{\text{pre}}(q) \leq \frac{q}{2^{1024}}, \quad \mathbf{Adv}_{\text{C-Hash-1024}}^{\text{2nd-pre}}(q) \leq \frac{O(q(l + l'))}{2^n} + \frac{q}{2^{1024}}.$$

For $n = 1472$ and $l, l' \leq 2^{56}$, both bounds remain negligible for $q \leq 2^{1024}$. Thus C-Hash-1024 achieves full 1024-bit preimage and second-preimage security even though the generic indifferenciability argument only certifies 708-bit collision resistance.

Comparison with Rocket-1 Compared with Rocket-1 (CTR-Perm), the quartic term $\frac{9q^4l}{2^{3(b-w)/2+2}}$ and the cross-round meet-in-the-middle terms vanish entirely. The feedforward XOR in CTR-Func eliminates the backward-query vulnerability that enabled the high-multiplicity backward-backward collision strategy, reducing the dominant VIL term from $O(q^4/2^{3n/2})$ to $O(q^2/2^n)$.

The remaining limitation is the information-theoretic birthday bound $O(q^2l/2^n)$, which is inherent to any n -bit iterated state and sets the generic indifferenciability ceiling at $q \sim 2^{n/2}/\sqrt{l}$. For $n = 1472$ this yields $q \leq 2^{708}$, significantly surpassing the 512-bit indifferenciability guarantee of C-Hash-512 but naturally capped below the 1024-bit output length of C-Hash-1024. The latter’s full 1024-bit preimage guarantee is recovered via the direct VIL-FIL composition analysis in Appendix C.8, which bypasses the generic collision barrier.

C.6 Turning Indifferentiability to Hash Security

The indifferentiability framework guarantees that a construction inherits all information-theoretic security properties of a random oracle. Andreeva *et al.* [1] established the generic reductions from indifferentiability to the classical hash security notions. For any adversary making at most q queries, the following inequalities hold:

$$\begin{aligned}\mathbf{Adv}_H^{\text{pre}}(q) &\leq \mathbf{Adv}_H^{\text{indiff}}(q) + \frac{q}{2^{\text{hashlen}}}, \\ \mathbf{Adv}_H^{2\text{nd-pre}}(q) &\leq \mathbf{Adv}_H^{\text{indiff}}(q) + \frac{q}{2^{\text{hashlen}}}, \\ \mathbf{Adv}_H^{\text{coll}}(q) &\leq \mathbf{Adv}_H^{\text{indiff}}(q) + \frac{q(q-1)}{2^{\text{hashlen}+1}}.\end{aligned}$$

These reductions are tight: the additional terms are exactly the success probabilities of generic attacks against a random oracle. Consequently, once the indifferentiability advantage is bounded, the concrete preimage, second-preimage, and collision resistances follow immediately.

Rocket-1 (CTR-Perm) Substituting the indifferentiability bound of Rocket-1 from Appendix C.4.3 yields:

$$\begin{aligned}\mathbf{Adv}_{\text{Rocket-1}}^{\text{pre}}(q) &\leq \frac{19q^2l}{2^{b-w}} + \frac{3(z-1)q^2}{2^{b-w}} + \frac{9q^4l}{2^{3(b-w)/2+2}} + \frac{O(q^3)}{2^b} + \frac{q}{2^{\text{hashlen}}}, \\ \mathbf{Adv}_{\text{Rocket-1}}^{2\text{nd-pre}}(q) &\leq \frac{19q^2l}{2^{b-w}} + \frac{3(z-1)q^2}{2^{b-w}} + \frac{9q^4l}{2^{3(b-w)/2+2}} + \frac{O(q^3)}{2^b} + \frac{q}{2^{\text{hashlen}}}, \\ \mathbf{Adv}_{\text{Rocket-1}}^{\text{coll}}(q) &\leq \frac{19q^2l}{2^{b-w}} + \frac{3(z-1)q^2}{2^{b-w}} + \frac{9q^4l}{2^{3(b-w)/2+2}} + \frac{O(q^3)}{2^b} + \frac{q^2}{2^{\text{hashlen}+1}}.\end{aligned}$$

For $l \geq z$, the linear term in z is absorbed into the $q^2l/2^{b-w}$ asymptotic. The dominant feature at moderate query regimes is the quartic term $9q^4l/2^{3(b-w)/2+2}$, inherited from the backward-query vulnerability of CTR-Perm; at the ceiling it is superseded by the cm-pre term $O(q^3)/2^b$ from the USMA attack, which caps the indifferentiability of Rocket-1 at $b/3 = 512$ bits. The generic reduction therefore certifies preimage and second-preimage resistance up to $q \leq 2^{512}$, coinciding with the 512-bit targets of C-Hash-512. The direct analysis of Appendix C.7 independently establishes the full 512-bit level without relying on the indifferentiability ceiling. Collision resistance is governed by the 512-bit digest's birthday bound at 256 bits and is unaffected by the cm-pre term.

Rocket-1f (CTR-Func) Substituting the indifferentiability bound of Rocket-1f from Appendix C.5.2 yields:

$$\begin{aligned}
\text{Adv}_{\text{Rocket-1f}}^{\text{pre}}(q) &\leq \frac{O(q^2l)}{2^{b-w}} + \frac{3q}{2^{(b-w)/2}} + \frac{q}{2^{\text{hashlen}}}, \\
\text{Adv}_{\text{Rocket-1f}}^{\text{2nd-pre}}(q) &\leq \frac{O(q^2l)}{2^{b-w}} + \frac{3q}{2^{(b-w)/2}} + \frac{q}{2^{\text{hashlen}}}, \\
\text{Adv}_{\text{Rocket-1f}}^{\text{coll}}(q) &\leq \frac{O(q^2l)}{2^{b-w}} + \frac{3q}{2^{(b-w)/2}} + \frac{q^2}{2^{\text{hashlen}+1}}.
\end{aligned}$$

For the concrete parameters $n = b - w = 1472$ and $l \leq 2^{56}$, the dominant indistinguishability term $O(q^2l)/2^n$ reaches 1 when $q \approx 2^{708}$. The birthday term $3q/2^{n/2}$ is then $O(2^{-28})$ and negligible. Consequently, the generic indistinguishability guarantee holds for $q \leq 2^{708}$.

For **C-Hash-512** ($\text{hashlen} = 512$), the random-oracle collision term $q^2/2^{513}$ reaches 1 at $q \approx 2^{256}$, so the generic reduction yields 256-bit collision resistance; the 512-bit preimage and second-preimage guarantees rest on the direct analysis of Appendix C.7.

For **C-Hash-1024** ($\text{hashlen} = 1024$), the random-oracle preimage term $q/2^{1024}$ is negligible for $q \leq 2^{708}$, but the indistinguishability term reaches 1 at $q \approx 2^{708}$. Hence the generic reduction certifies 708-bit preimage and second-preimage security through indistinguishability. Collision resistance is governed by the 1024-bit birthday bound, i.e., 512 bits. A direct analysis of the VIL-FIL composition (Appendix C.8) strengthens the preimage and second-preimage bounds to full 1024 bits for $q \leq 2^{1024}$, independent of the generic indistinguishability ceiling.

C.7 Direct Proofs for the Preimage of Rocket-JH on CTR-Perm

The generic indistinguishability reduction of the preceding section certifies the classical security notions of Rocket-1 up to the $b/3 = 512$ -bit ceiling imposed by the cm-pre (USMA) term. For **C-Hash-512** ($\text{hashlen} = 512$), the targeted 512-bit preimage and second-preimage levels coincide with this ceiling but lie far below the $n/2$ -bit VIL barrier. This appendix shows, by a direct analysis of the two-stage composition $F \circ H$ over CTR-Perm, that both targets are met with full margin. The decisive observation is that the chosen-multiple-preimage amplification underlying USMA is unavailable in the preimage and second-preimage games: there, the target digest is either externally sampled at random or fixed by a challenge message, so the adversary cannot endow the finalization inputs with the shared algebraic structure that USMA exploits.

C.7.1 Direct Preimage Analysis Let $y \in \{0, 1\}^{512}$ be the uniformly sampled target digest. Write the construction as

$$\text{Rocket-1}(M) = \text{LMB}\left(512, F(H(M))\right),$$

where $H = \text{R1CP}$ is the VIL component and F is the single-round CTR-Perm finalization ($z = 1$, $h = 736$, $n = b - w = 1472$).

Route 1: forward evaluation (generic). Each evaluation of Rocket-1 on a chosen message yields, for a fresh VIL output, a digest that is uniform over $\{0, 1\}^{512}$: the finalization F maps the n -bit VIL output to a 736-bit block r_1 that is uniform within the birthday bound of P , and truncation to 512 bits preserves uniformity. Hence

$$\Pr[\text{forward hit}] \leq \frac{q}{2^{512}}.$$

Route 2: finalization inversion. The adversary may instead try to invert F on y . Querying $P^{-1}(y \parallel \rho)$ for arbitrary $\rho \in \{0, 1\}^{b-512}$ returns an input $x \parallel \text{ctr}$ with ctr uniform over $\{0, 1\}^w$. The pair is useful only if $\text{ctr} = \text{ctr}_{\text{fil}}$, which holds with probability 2^{-w} ; in that case x is a valid finalization input with $\text{LMB}(512, F(x)) = y$. Each candidate VIL-state therefore costs $2^w = 2^{64}$ inverse queries on average, and with q queries the adversary collects at most $Q \leq q/2^w$ candidate states x^* .

Crucially, these candidates are statistically independent and unstructured: because y is random, the adversary cannot select them to share a common half or to lie inside a subspace, which is exactly the structural freedom that the cm-pre (USMA) attack requires. Chaining any one candidate x^* back to an initial value through the VIL component is the (multi-target) preimage problem on unstructured n -bit states, covered by the rm-pre analysis of R1CP: with $Q \leq q$ unstructured candidates, the success probability is bounded by

$$\Pr[\text{inversion reach}] \leq \mathbf{Adv}_{\text{R1CP}}^{\text{rm-pre}}(Q, q) \leq \frac{9q^2l}{2^n},$$

which is below 2^{-388} for $q \leq 2^{512}$, $n = 1472$, and $l \leq 2^{56}$.

Route 3: chosen-target (USMA-type) amplification. The cm-pre barrier $O(q^3)/2^n$ established in Appendix C does not transfer to the random-target preimage game: USMA’s amplification factor comes from manufacturing target states that share one common half, whereas the finalization-inversion candidates of Route 2 are full-entropy states outside the adversary’s structural control. With the targets unstructured, the optimal meet-in-the-middle strategy focuses on a single candidate, and no multi-target amortization exists—this is precisely the regime covered by the rm-pre bound.

Conclusion. The optimal preimage strategy is the generic forward one:

$$\mathbf{Adv}_{\text{Rocket-1}}^{\text{pre}}(q) \leq \frac{q}{2^{512}} + \frac{9q^2l}{2^n},$$

where the second term is below 2^{-388} for $q \leq 2^{512}$, $n = 1472$, and $l \leq 2^{56}$. Hence C-Hash-512 achieves full 512-bit preimage resistance.

C.7.2 Direct Second-Preimage Analysis Given a challenge message M' of length l' and its digest y , the adversary seeks $M \neq M'$ with $\text{Rocket-1}(M) = y$. Two mutually exclusive strategies arise.

Strategy 1: iterative collision. Find M whose chain reaches the final VIL state of M' ; the finalization then collides automatically. Because the round functions are position-dedicated (RoundCounter binds each block to its index), the adversary cannot transplant blocks across positions. Reaching the final state is the single-target preimage problem on an n -bit state with bound $t^2 q^2 l / 2^n$; reaching any intermediate state h'_i (and appending the known suffix of M') costs the same per position, and the union over $i' \leq l'$ contributes a factor l' . Both are bounded by

$$\frac{t^2 q^2 l l'}{2^n} \leq \frac{9 q^2 l l'}{2^n},$$

which is below 2^{-332} for $q \leq 2^{512}$, $n = 1472$, and $l, l' \leq 2^{56}$. We remark that long-message second-preimage attacks in the Kelsey–Schneier family rely on reusing one compression function at arbitrary positions to build expandable messages; the pairwise distinct round functions of Rocket-JH bind every block to its round index, eliminating this class of attacks.

Strategy 2: finalization collision. Find M with $H(M) \neq H(M')$ but equal digest. Each forward evaluation yields a uniform 512-bit digest, and each finalization-inversion candidate (costing 2^w) is likewise a uniform 512-bit value whose subsequent chaining is covered by the rm-pre bound of Route 2. Hence

$$\Pr[\text{finalization collision}] \leq \frac{q}{2^{512}} + \frac{9 q^2 l}{2^n}.$$

Overall bound. Combining both strategies:

$$\mathbf{Adv}_{\text{Rocket-1}}^{\text{2nd-pre}}(q) \leq \frac{9 q^2 l l'}{2^n} + \frac{9 q^2 l}{2^n} + \frac{q}{2^{512}}.$$

For C-Hash-512 ($n = 1472$, $\text{hashlen} = 512$) and $l, l' \leq 2^{56}$, the VIL terms are below 2^{-332} for $q \leq 2^{512}$, so the overall advantage is dominated by $q/2^{512}$, establishing full 512-bit second-preimage resistance.

C.8 Direct Proofs for the Preimage of Rocket-JH on CTR-Func

The indistinguishability bounds derived in Appendix C.5.2 guarantee that Rocket-1f inherits the structural properties of a random oracle up to a certain query complexity. However, for concrete parameter choices such as those in C-Hash-1024 ($n = 1472$, $\text{hashlen} = 1024$), a direct analysis of the two-stage composition $F' \circ H'$ yields even tighter preimage and second-preimage guarantees. The intuition is that the VIL compression $H' = \text{R1CF}$ already achieves high preimage resistance by itself, while the FIL finalization F' behaves as an independent random function; the composition therefore inherits the stronger of the two security levels.

C.8.1 Direct Preimage Analysis Given a target hash value $y \in \{0, 1\}^{\text{hashlen}}$, an adversary $\mathcal{A}$ must find a message M such that

$$\text{Rocket-1f}(M) = F'(H'(M)) = y.$$

The construction is the composition of the VIL compression $H' = \text{R1CF}$ and the single-round CTR-Func finalization F' with $z = 1$ and effective width $n = b - w$.

FIL stage. The finalization F' computes

$$F'(x) = \text{LMB}(\text{hashlen}, P(x \parallel \text{ctr}_{\text{fil}})) \oplus \text{LMB}(\text{hashlen}, x),$$

where $x \in \{0, 1\}^n$ and $\text{hashlen} \leq n$. Because P is a random permutation, for any fresh x the n -bit value $\text{LMB}(n, P(x \parallel \text{ctr}_{\text{fil}}))$ is uniformly random over $\{0, 1\}^n$. Consequently, its hashlen -bit prefix $\text{LMB}(\text{hashlen}, P(x \parallel \text{ctr}_{\text{fil}}))$ is uniformly random over $\{0, 1\}^{\text{hashlen}}$, and the XOR with the fixed prefix $\text{LMB}(\text{hashlen}, x)$ preserves this uniformity. Thus $F'(x)$ is uniformly distributed over $\{0, 1\}^{\text{hashlen}}$ for every distinct x .

The adversary may attack F' in two directions:

- **Forward queries.** The adversary selects an arbitrary x (either by computing $x = H'(M)$ for some M or by choosing x directly) and evaluates $F'(x)$. Each such evaluation yields an independent uniform value in $\{0, 1\}^{\text{hashlen}}$, so the probability of hitting the target y is exactly $2^{-\text{hashlen}}$.
- **Backward queries.** The adversary attempts to invert F' on y by querying P^{-1} . For any b -bit value v , the inverse query returns $x \parallel \text{ctr}$ with x uniformly random over $\{0, 1\}^n$. If $\text{ctr} = \text{ctr}_{\text{fil}}$, the condition $\text{LMB}(\text{hashlen}, v) \oplus \text{LMB}(\text{hashlen}, x) = y$ holds with probability exactly $2^{-\text{hashlen}}$ because x is uniform and independent of v . Hence each backward query produces a valid preimage x of y under F' with probability at most $2^{-\text{hashlen}}$.

Composition. Because $F'(x)$ is a uniform random function from $\{0, 1\}^n$ to $\{0, 1\}^{\text{hashlen}}$, the distribution of $F'(H'(M))$ is independent of the internal structure of H' . The adversary's optimal strategy is therefore to maximize the number of distinct inputs to F' , whether by evaluating H' on arbitrary messages or by inverting F' directly. Each distinct query to F' (forward or backward) succeeds with probability $2^{-\text{hashlen}}$, and the q queries are independent. The overall preimage advantage is bounded by

$$\text{Adv}_{\text{Rocket-1f}}^{\text{pre}}(q) \leq \frac{q}{2^{\text{hashlen}}}.$$

For C-Hash-1024 with $\text{hashlen} = 1024$, this establishes full 1024-bit preimage resistance for all $q \leq 2^{1024}$. The bound is dominated entirely by the FIL finalization; the VIL compression H' does not introduce any weakness because its n -bit output space is larger than the target length ($n = 1472 > 1024$).

C.8.2 Direct Second-Preimage Analysis Given a challenge message M' of length l' with hash output y , an adversary seeks $M \neq M'$ such that $\text{Rocket-1f}(M) = y$. Two mutually exclusive strategies arise:

Strategy 1: Iterative collision. Find $M \neq M'$ with $H'(M) = H'(M')$. Then $F'(H'(M)) = F'(H'(M')) = y$ automatically. This reduces to the second-preimage resistance of R1CF. From the direct analysis of R1CF in Appendix C.5.1,

$$\text{Adv}_{\text{R1CF}}^{\text{2nd-pre}}(q) \leq \frac{O(q(l + l'))}{2^n} + \frac{O(q^2(l + l'))}{2^{3n/2}}.$$

For $n = 1472$ and $l, l' \leq 2^{56}$, the VIL terms are of order 2^{-103} and negligible compared to the $q/2^{1024}$ term.

Strategy 2: Finalization collision. Find $M \neq M'$ with $H'(M) \neq H'(M')$ but $F'(H'(M)) = F'(H'(M')) = y$. Since F' is a uniform random function, for any distinct $x \neq x'$ the collision probability is

$$\Pr[F'(x) = F'(x')] = 2^{-\text{hashlen}}.$$

With q attempts to find such a pair,

$$\Pr[\text{finalization collision}] \leq \frac{q}{2^{\text{hashlen}}}.$$

Overall bound. Combining both strategies:

$$\text{Adv}_{\text{Rocket-1f}}^{\text{2nd-pre}}(q) \leq \frac{O(q(l+l'))}{2^n} + \frac{O(q^2(l+l'))}{2^{3n/2}} + \frac{q}{2^{\text{hashlen}}}.$$

For C-Hash-1024 ($n = 1472$, $\text{hashlen} = 1024$), the first two terms are below 2^{-103} for $q \leq 2^{1024}$ and $l, l' \leq 2^{56}$, while the third term is exactly $q/2^{1024}$. Consequently, the overall advantage is bounded by $q/2^{1024} + 2^{-103}$, establishing full 1024-bit second-preimage resistance.

C.9 Indifferentiability of LDXOF and LDRH

CTR-Perm as the Cryptographic Foundation. Recall that CTR-Perm derives domain-separated function instances from a single public permutation $P: \{0, 1\}^b \rightarrow \{0, 1\}^b$ by appending a w -bit counter ctr to the n -bit payload. Within the birthday bound $q \ll 2^{b/2}$, forward evaluations on distinct inputs $x \parallel \text{ctr}$ yield outputs that are indistinguishable from uniform random values. Consequently, for each fixed counter ctr , the mapping

$$x \mapsto P_{\text{ctr}}(x) = \text{LMB}(n, P(x \parallel \text{ctr}))$$

behaves as an independent random function.

The backward-query interface is also preserved: the adversary may query P^{-1} on arbitrary b -bit values. However, because the w -bit counter acts as a domain separator, a backward query returns a valid n -bit payload for a specific ctr with probability 2^{-w} . This controlled multiplicity is precisely the setting analyzed by Bhaumik *et al.* [5] for the XOR-of-permutations paradigm in the public-permutation model.

LDXOF Indifferentiability. LDXOF instantiates the XORP (XOR of Permutations) core via two domain-separated CTR-Perm calls with counters ctr_0 and ctr_i for $i = 1, \dots, l$, adapting Theorem 10 of Bhaumik *et al.* [5] to the CTR-Perm framework. We replace their instance-count parameter with the LDXOF block parameter l .

Theorem 4 (Indifferentiability of LDXOF on CTR-Perm). *Let $P: \{0, 1\}^b \rightarrow \{0, 1\}^b$ be a public random permutation with $b = n + w$, $n = 1472$, $w = 64$. Let LDXOF be instantiated via two domain-separated CTR-Perm calls with distinct counters per block index $i \in [0, l]$. Then for any distinguisher making at most q queries with $9n \leq 2q \leq 2^n - 1$,*

$$\text{Adv}_{\text{LDXOF}}^{\text{indiff}}(q) \leq \frac{9l^2 q \sqrt{nq}}{2^n} + \frac{48l^4 q^3}{2^{2n}} + \frac{2l^2 q}{2^n} + \frac{l^2}{2^n}.$$

The first term dominates for all relevant query regimes. Equating this dominant term to 1 yields

$$q \approx \frac{2^{2n/3}}{(9l^2\sqrt{n})^{2/3}}, \quad \text{so that} \quad \log_2 q \approx \frac{2n}{3} - \frac{1}{3}\log_2 n - O(\log_2 l).$$

For constant l , this gives approximately 972-bit indistinguishability security.

With $n = 1472$, this bound places the construction in the beyond-birthday-bound regime, delivering approximately 972-bit indistinguishability security for any constant output length $l = O(1)$. This comfortably exceeds the 512-bit and 708-bit indistinguishability targets of C-Hash-512 and C-Hash-1024, respectively.

LDRH as a Special Case. LDRH corresponds to LDXOF restricted to a single output block ($l = 1$). In this case the XORP construction collapses to the sum of two CTR-Perm instances $P_{\text{ctr}_1}(x) \oplus P_{\text{ctr}_2}(x)$.

Corollary 1 (Indistinguishability of LDRH on CTR-Perm). *Setting $l = 1$ in Theorem 4, the LDRH construction satisfies*

$$\text{Adv}_{\text{LDRH}}^{\text{indiff}}(q) \leq \frac{9\sqrt{n}q^{3/2}}{2^n} + \frac{48q^3}{2^{2n}} + \frac{2q+1}{2^n}.$$

The first term dominates for all relevant query regimes. Equating this dominant term to 1 yields the query threshold

$$q \approx \frac{2^{2n/3}}{(9\sqrt{n})^{2/3}}, \quad \text{so that} \quad \log_2 q \approx \frac{2n}{3} - \frac{1}{3}\log_2 n - O(1).$$

For $n = 1472$ the effective security is therefore approximately 972 bits. Consequently, LDRH achieves roughly $\frac{2n}{3} - \frac{1}{3}\log_2 n - O(1)$ -bit indistinguishability security.

Lane Isolation and Independence. LDXOF and LDRH occupy dedicated lane identifiers that are cryptographically isolated from the standard multi-block hash lanes of the C-Hash family:

- C-Hash-512: LDRH uses `LaneID = 0x03`, LDXOF uses `LaneID = 0x04`.
- C-Hash-1024: LDRH uses `LaneID = 0x83`, LDXOF uses `LaneID = 0x84`.

These values are distinct from the VIL compression lanes (`0x01, 0x81`) and FIL finalization lanes (`0x02, 0x82`). Because the 8-bit `LaneID` field forms the most-significant byte of the w -bit counter, any query to an LDXOF or LDRH lane cannot collide with a query to a Rocket-JH lane in the underlying permutation P . Consequently, the security analyses of the two systems are completely decoupled: the indistinguishability bound of Rocket-JH is unaffected by the presence of LDXOF or LDRH lanes, and vice versa.

E Statistical Test Results of C Hash

To empirically validate the pseudorandomness of the C Hash family, we conducted the GM/T 0005 randomness statistical test suite on the output bit streams produced by C-Hash-512 and C-Hash-1024. For each parameter set, 1000 binary sequences of 1,000,000 bits each were generated and evaluated under two adversarial input models: a *low-Hamming-weight* message model and a *high-Hamming-weight* message model, which stress the diffusion behavior for sparse and dense inputs, respectively. A sequence is regarded as passing an item if its second-level significance value exceeds the threshold of 98.05607204%; the reported figures are the percentage of the 1000 sequences that pass each item.

E.1 Randomness Statistical Test Results of C-Hash-512

The detailed per-item results of C-Hash-512 are shown in Table 11.

Table 11: Randomness statistical test results of C-Hash-512

Test Item	Low Hamming Weight (%)	High Hamming Weight (%)
Monobit Frequency	99.30	98.70
Block Frequency (10000)	99.00	98.60
Poker (4)	98.80	98.80
Overlapping Subsequence (5)-1	99.00	99.30
Overlapping Subsequence (5)-2	99.20	99.00
Runs Total	99.30	99.00
Runs Distribution	98.80	98.60
Binary Derivative (7)	99.00	98.80
Autocorrelation (2)	98.90	98.40
Matrix Rank	99.30	99.20
Cumulative Sums (Forward)	99.30	98.60
Cumulative Sums (Backward)	99.20	98.40
Approximate Entropy (5)	99.50	98.40
Linear Complexity (500)	99.00	99.20
Universal Statistical (7)(1280)	99.30	99.50
Discrete Fourier Transform	98.60	98.90

E.2 Randomness Statistical Test Results of C-Hash-1024

The detailed per-item results of C-Hash-1024 are shown in Table 12.

Table 12: Randomness statistical test results of C-Hash-1024

Test Item	Low Hamming Weight (%)	High Hamming Weight (%)
Monobit Frequency	99.00	99.30
Block Frequency (10000)	99.10	99.20
Poker (4)	98.50	99.40
Overlapping Subsequence (5)-1	99.00	99.40

Randomness statistical test results of C-Hash-1024 (continued)

Test Item	Low Hamming	High Hamming
	Weight (%)	Weight (%)
Overlapping Subsequence (5)-2	98.70	99.50
Runs Total	99.00	99.30
Runs Distribution	98.20	98.70
Binary Derivative (7)	98.90	98.50
Autocorrelation (2)	98.80	99.00
Matrix Rank	99.50	99.00
Cumulative Sums (Forward)	99.10	99.40
Cumulative Sums (Backward)	99.10	99.30
Approximate Entropy (5)	99.00	99.60
Linear Complexity (500)	99.60	99.10
Universal Statistical (7)(1280)	98.70	98.60
Discrete Fourier Transform	99.30	99.40